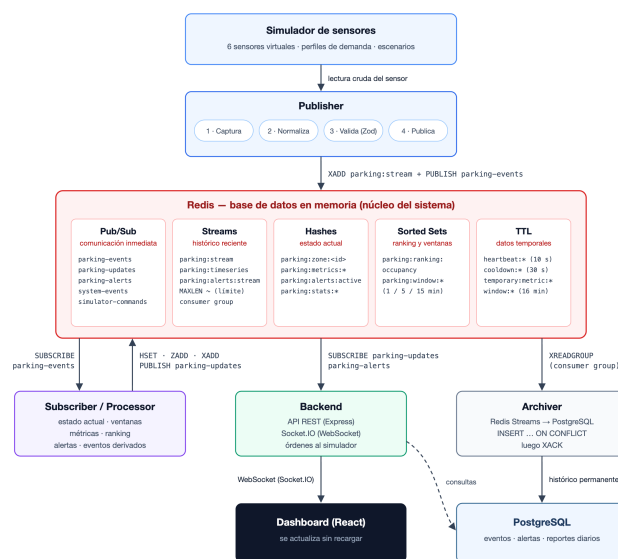


UPTC SMART PARKING

Sistema distribuido de monitoreo de estacionamientos en tiempo real con Redis

Documento del proyecto: explicación completa,
arquitectura y funcionamiento

Actividad integradora · Grupo 4 — Estacionamiento inteligente para carros y motocicletas



Integrantes: _____

Docente: _____

Sogamoso, septiembre de 2026

Contenido

Cómo usar este documento	3
1. Resumen del proyecto	4
2. El problema y los objetivos	6
3. Conceptos clave antes de empezar	8
4. Arquitectura general	10
5. Cómo funciona: el recorrido completo de un evento	13
6. El simulador de sensores	19
7. Redis en detalle	24
8. El Subscriber y el Processor	31
9. Métricas	33
10. Alertas	35
11. Backend: API REST y tiempo real	38
12. El dashboard	40
13. PostgreSQL: el histórico permanente	46
14. Tolerancia a fallos	48
15. Estructura del código	49
16. Instalación y ejecución	51
17. Pruebas y resultados	53
18. Guía para la exposición	55
19. Preguntas de sustentación	58
20. Limitaciones, dificultades y mejoras	61
21. Conclusiones	62
Anexo A. Glosario	63
Anexo B. Comandos Redis útiles para la demostración	64
Anexo C. Cumplimiento de los requisitos	65
Anexo D. Documentación complementaria en el repositorio	66

Cómo usar este documento

Este documento explica de principio a fin el sistema **UPTC Smart Parking**: qué problema resuelve, cómo está construido, qué hace cada componente, cómo viaja la información y cómo se demuestra en vivo. Está pensado para dos usos al mismo tiempo:

- **Entregarlo al docente** como descripción completa y formal del proyecto (arquitectura, diseño de Redis, simulador, métricas, alertas, pruebas y resultados).
- **Estudiarlo para la exposición**: cada capítulo explica los conceptos en lenguaje sencillo, con ejemplos reales tomados del sistema en ejecución, y termina con lo que conviene decir o mostrar.

A lo largo del texto aparecen cuatro tipos de recuadro:

EN PALABRAS SIMPLES

Explicación con palabras cotidianas y analogías, para entender el concepto antes de los detalles técnicos.

IDEA CLAVE

La idea central que todo el grupo debe poder repetir sin leer.

DETALLE TÉCNICO

Datos precisos de implementación: comandos Redis, nombres de archivos, parámetros.

PARA LA EXPOSICIÓN

Qué mostrar o decir durante la sustentación y cómo responder preguntas del jurado.

Todos los ejemplos (eventos, comandos, registros de log y capturas de pantalla) salen del sistema real funcionando en Docker el 22 de septiembre de 2026. Las zonas, capacidades y perfiles de demanda son **valores de demostración**: no representan los parqueaderos físicos de la universidad.

1. Resumen del proyecto

UPTC Smart Parking es un **sistema distribuido de monitoreo en tiempo real** para los parqueaderos de carros y motocicletas de la UPTC Seccional Sogamoso. Unos sensores simulados detectan cada vehículo que entra o sale de una zona; ese hecho se convierte en un **evento** que viaja por una cadena de servicios independientes y termina actualizando un tablero web **sin que nadie recargue la página**.

La pieza central es **Redis**, una base de datos en memoria que en este proyecto cumple cinco papeles: mensajero entre servicios (Pub/Sub), bitácora de eventos recientes (Streams), ficha del estado actual de cada zona (Hashes), ranking de ocupación (Sorted Sets) y almacén de datos que caducan solos (TTL). **PostgreSQL** complementa a Redis guardando el histórico permanente para reportes.

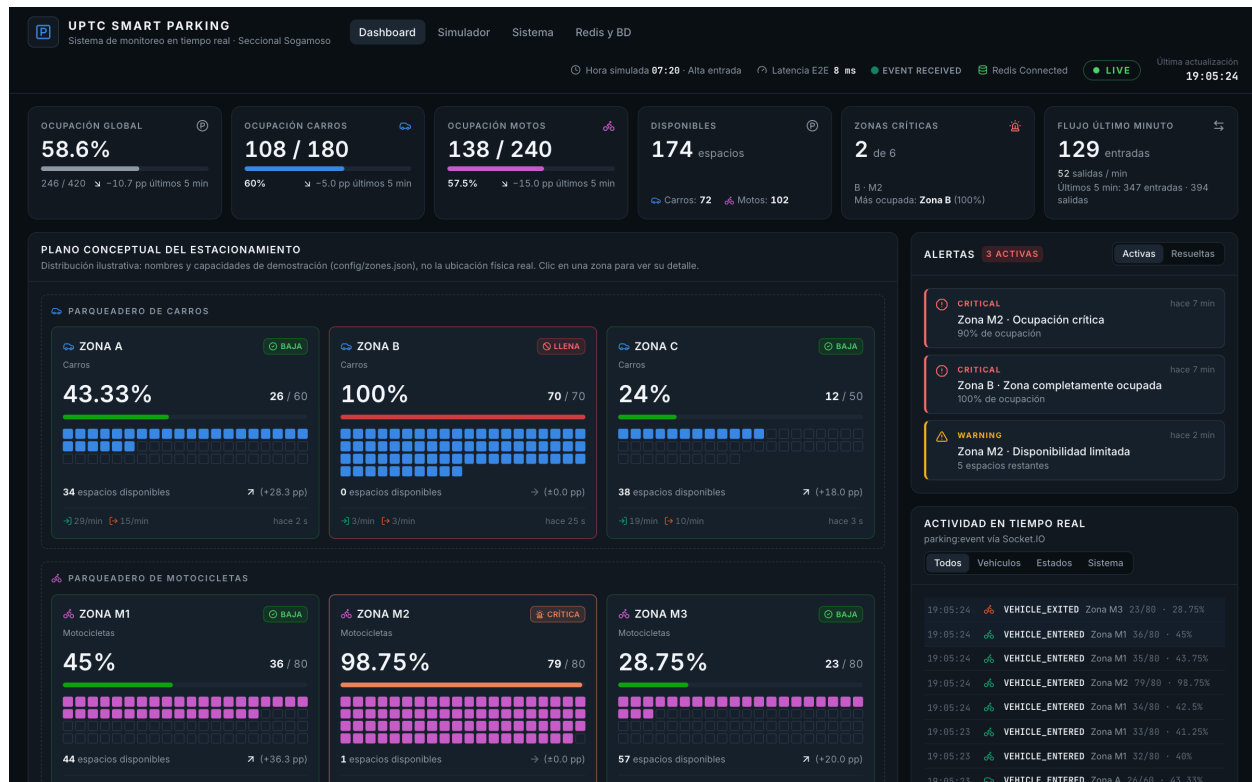


Figura 1. Dashboard en funcionamiento: Zona B llena (roja), Zona M2 en estado crítico (98,75 %), tres alertas activas a la derecha y la actividad llegando en tiempo real.

Indicador	Valor
Zonas monitoreadas	6 (3 de carros y 3 de motocicletas) · 420 espacios en total
Servicios independientes	6 contenedores: redis, publisher, processor, backend, archiver, frontend
Mecanismos de Redis utilizados	5: Pub/Sub, Streams, Hashes, Sorted Sets y TTL (el mínimo exigido era 2)
Métricas derivadas	11 (más ventanas de 1, 5 y 15 minutos)
Tipos de alerta	7, con escalado, histéresis, cooldown y recuperación
Gráficas en el dashboard	3 (evolución de ocupación, entradas vs salidas, ranking por zona)
Latencia de extremo a extremo	≈ 9–13 ms desde el sensor hasta el navegador
Tiempo de procesamiento por evento	≈ 4–6 ms

Indicador	Valor
Pruebas automatizadas	72 (68 unitarias + 4 de integración con Redis real), todas aprobadas
Ejecución	Un solo comando: docker compose up --build

IDEA CLAVE

Cuando un vehículo entra a una zona, el cambio viaja **Simulador → Publisher → Redis → Processor → Backend → WebSocket → Dashboard** en unos 13 milisegundos, y la tarjeta de la zona cambia sola en la pantalla.

Redis no se usa como una simple tabla: es el canal de comunicación, la memoria del estado actual y la bitácora del histórico reciente.

2. El problema y los objetivos

2.1 Descripción del problema

El control de ocupación de estacionamientos requiere conocer de manera rápida cuántos espacios se encuentran disponibles y cuáles zonas presentan alta demanda. En este proyecto se desarrolla un sistema de monitoreo en tiempo real para representar conceptualmente los estacionamientos de carros y motocicletas de la UPTC Seccional Sogamoso. La solución utiliza sensores simulados que generan eventos de entrada y salida, Redis para comunicación y almacenamiento temporal, un componente de procesamiento encargado de calcular métricas y alertas, y una aplicación web que permite visualizar dinámicamente el estado del sistema.

La dificultad no es guardar un número por zona. Lo difícil es que:

- el número **cambia continuamente** (decenas de veces por minuto en horas pico);
- **varios componentes** necesitan enterarse del cambio al mismo tiempo (cálculo de métricas, alertas, tablero, archivo histórico);
- hay que **detectar situaciones críticas** (zona casi llena, zona llena, crecimiento anormal) en el momento en que ocurren, sin avisar diez veces lo mismo;
- el sistema debe **seguir funcionando aunque una parte falle** (por ejemplo, si el procesador se reinicia no debe perder lo que pasó mientras estaba caído).

EN PALABRAS SIMPLES

Una página que consulta una base de datos cada cierto tiempo (por ejemplo cada 10 segundos) siempre va atrasada y hace trabajo inútil cuando nada cambió. En cambio, en una arquitectura orientada a eventos, **cada cambio avisa por sí mismo** a quien le interesa, en el instante en que ocurre.

2.2 Objetivo general

Diseñar e implementar un sistema distribuido de monitoreo en tiempo real que simule sensores de estacionamiento, comunique sus eventos mediante Redis, mantenga el estado actual y el histórico reciente, calcule métricas derivadas, detecte situaciones críticas y actualice una interfaz web automáticamente.

2.3 Objetivos específicos

1. Usar Redis como base de datos en memoria con cinco mecanismos: Pub/Sub, Streams, Hashes, Sorted Sets y TTL.
2. Construir un simulador de sensores con continuidad temporal, perfiles de demanda por hora y escenarios forzables.
3. Separar Publisher, Subscriber/Processor, Backend y Dashboard en procesos independientes.
4. Normalizar y validar cada evento con un esquema común (Zod) antes de publicarlo.
5. Calcular métricas derivadas: ocupación por zona, global y por tipo de vehículo; disponibilidad; entradas y salidas por ventana de tiempo; variación; tendencia; ranking; zonas críticas.
6. Generar alertas sin duplicados, con escalado, histéresis, cooldown y recuperación.
7. Diferenciar explícitamente el estado actual (Hashes) del histórico reciente (Streams).
8. Transmitir los cambios al navegador por WebSocket y medir la latencia de extremo a extremo.

9. Controlar el crecimiento de la información almacenada (MAXLEN, TTL y retención).
10. Tolerar la desconexión de Redis, del Processor y del navegador, con reconexión automática.
11. Conservar el histórico permanente en PostgreSQL.
12. Documentar la arquitectura y garantizar una ejecución reproducible con Docker Compose.

2.4 Alcance y supuestos

El sistema SÍ contempla	El sistema NO pretende representar
Simulación de 6 zonas de carros y motocicletas Entradas, salidas, ocupación y disponibilidad Métricas y alertas en tiempo real Estado actual e histórico reciente (Redis) Histórico permanente (PostgreSQL) Visualización web con actualización en vivo Panel para forzar escenarios durante la demostración	Sensores físicos reales Capacidades reales de los parqueaderos Coordenadas reales (el campo location va en null) Placas o matrículas de vehículos Identidad de los conductores

Los nombres, capacidades y parámetros de demanda están en archivos de configuración (`config/zones.json`, `config/thresholds.json`, `config/simulation.json`) y pueden cambiarse **sin modificar el código**.

3. Conceptos clave antes de empezar

Estos conceptos aparecen en todo el documento. Vale la pena dominarlos antes de ver la arquitectura.

Base de datos en memoria

Una base de datos que guarda la información en la **memoria RAM** en lugar del disco. Leer o escribir en RAM toma microsegundos, por eso es ideal para datos que cambian muy rápido. A cambio, la RAM es más limitada y los datos pueden perderse si no se configura persistencia. **Redis** es la base de datos en memoria más usada del mundo.

Evento

Un registro de **algo que pasó**: "entró un vehículo a la Zona M2 a las 18:59:04 y ahora hay 73 de 80 ocupados". Un evento es inmutable (no se edita) y lleva toda la información necesaria para entender el hecho. En el proyecto se representa como un objeto JSON.

Arquitectura orientada a eventos

Forma de construir sistemas donde los componentes **no se llaman entre sí directamente**: uno publica que algo pasó y los interesados reaccionan. Así cada componente puede cambiar, reiniciarse o escalarse sin afectar a los demás.

EN PALABRAS SIMPLES

Es como un grupo de WhatsApp de vigilancia: el guarda de la entrada escribe "entró una moto a M2" y todos los que están en el grupo se enteran al instante. El guarda no necesita saber quiénes leen el mensaje ni qué hará cada uno con él.

Publisher y Subscriber

El **Publisher** (publicador) es quien emite los eventos. El **Subscriber** (suscriptor) es quien se inscribe a un canal para recibirlos. En el proyecto, el Publisher publica cada entrada o salida en el canal **parking-events** y el Subscriber del Processor la recibe.

Estado actual vs. histórico

El **estado actual** es la foto de cómo está una zona en este instante (73 de 80, 91,25 %, CRÍTICA). El **histórico** es la película de cómo llegó a ese valor (las entradas y salidas anteriores). En Redis el estado actual vive en un **Hash** y el histórico reciente en un **Stream**.

WebSocket

Una conexión que el navegador mantiene **abierta** con el servidor, de modo que el servidor puede enviarle datos cuando quiera, sin que el navegador pregunte. Es lo que permite que el tablero cambie sin recargar. Se usa mediante la librería **Socket.IO**, que además reconecta sola si la conexión se cae.

Contenedor (Docker)

Un paquete que incluye un programa con todo lo que necesita para ejecutarse. **Docker Compose** levanta todos los contenedores del proyecto con un solo comando, de forma idéntica en cualquier computador.

PARA LA EXPOSICIÓN

Si el jurado pregunta "¿por qué no un CRUD?": un CRUD sólo guarda y consulta datos cuando alguien lo pide. Aquí los datos se **empujan** en el momento en que ocurren, se procesan en el camino (métricas y alertas) y llegan solos a la pantalla.

4. Arquitectura general

El sistema está formado por seis servicios independientes que se comunican exclusivamente a través de Redis (y PostgreSQL para el histórico permanente). La figura muestra el panorama completo:

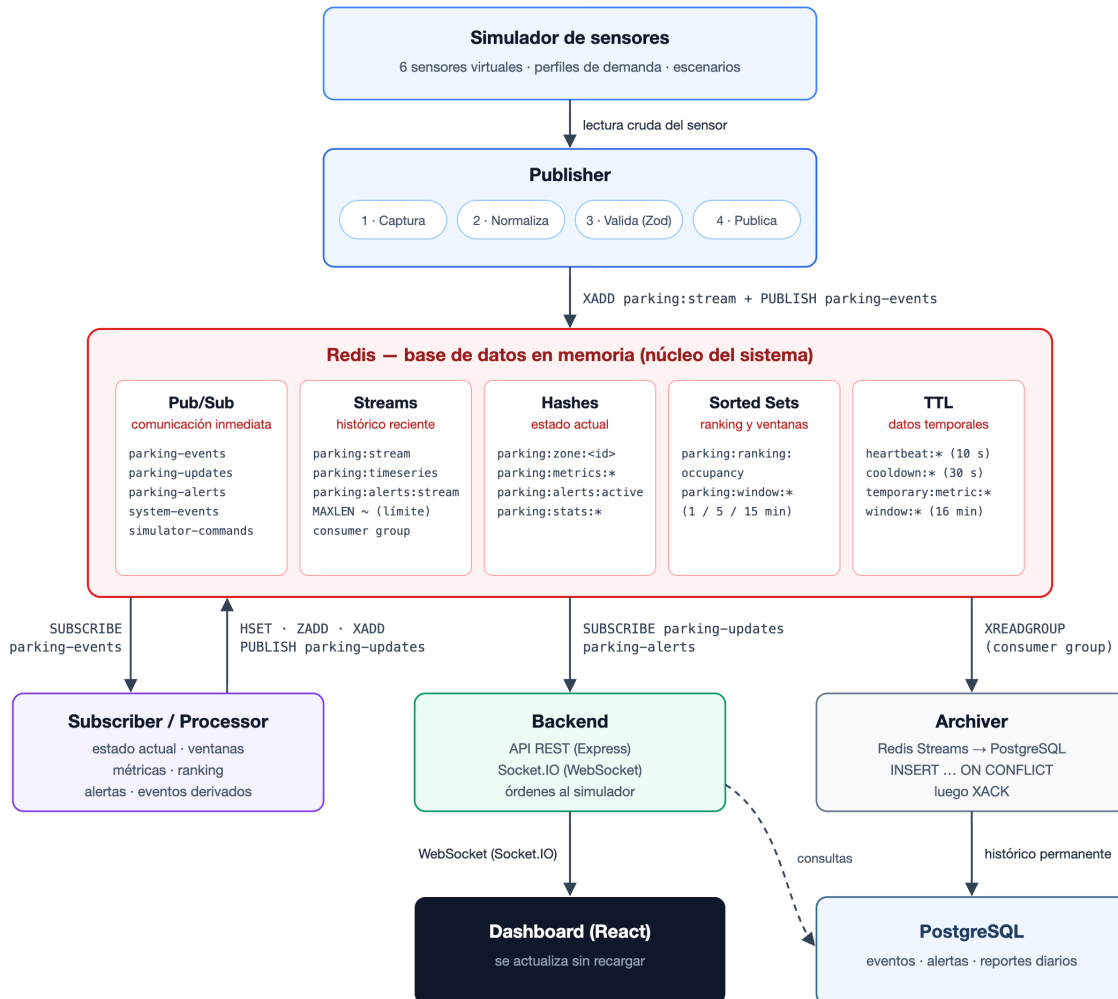


Figura 2. Arquitectura del sistema. Las flechas indican el sentido en que viaja la información y el comando Redis que la transporta.

4.1 Los componentes y su responsabilidad

Componente	Qué hace	Qué NO hace	Carpeta
Simulador	Imita los sensores de conteo de cada zona: decide cuándo entra o sale un vehículo según la hora y el escenario.	No calcula métricas ni se comunica con la interfaz.	<code>apps/publisher/src/simulation</code>
Publisher	Captura la lectura del sensor, la normaliza al formato común, la valida y la publica en Redis.	No calcula gráficas ni tiene lógica visual.	<code>apps/publisher/src/publisher</code>
Redis	Transporta los eventos, guarda el estado actual, el histórico reciente, el ranking y los datos temporales.	—	<code>docker/redis.conf</code>
Subscriber	Escucha el canal <code>parking-events</code> y entrega cada evento, en orden, al Processor. Recupera lo perdido.	No calcula nada.	<code>apps/processor/src/subscriber</code>
Processor	Calcula el estado actual, ventanas, métricas, ranking y alertas; publica el resultado.	No habla con el navegador.	<code>apps/processor/src</code>
Backend	API REST, puente Redis → WebSocket, estado del sistema, órdenes al simulador.	No calcula métricas: sólo las lee y difunde.	<code>apps/backend/src</code>
Dashboard	Muestra todo en tiempo real y permite controlar la simulación.	No calcula el estado: sólo lo presenta.	<code>apps/frontend/src</code>
Archiver	Copia el histórico de Redis a PostgreSQL con un consumer group.	No interviene en el tiempo real.	<code>apps/archiver/src</code>

IDEA CLAVE

Cada servicio tiene **una sola responsabilidad** y puede detenerse o reiniciarse sin tumbar a los demás. Ningún servicio llama a otro directamente: todos hablan con Redis.

4.2 Canales de comunicación (Pub/Sub)

Canal	Publica	Escucha	Contenido
<code>parking-events</code>	Publisher	Processor	Eventos crudos del sensor: VEHICLE_ENTERED y VEHICLE_EXITED
<code>parking-updates</code>	Processor	Backend	Estado procesado de la zona, métricas, eventos derivados y muestras cada 2 s
<code>parking-alerts</code>	Processor	Backend	Alertas levantadas (RAISED) y resueltas (RESOLVED)
<code>system-events</code>	Publisher	Backend	Simulación iniciada, pausada, cambio de escenario
<code>simulator-commands</code>	Backend	Publisher	Órdenes del panel de simulación (escenario, velocidad, pausa, entrada forzada...)

4.3 Tecnologías utilizadas

Tecnología	Versión	Para qué se usa
Node.js + TypeScript	22 / 5.9	Lenguaje de todos los servicios; un único modelo de tipos compartido
Redis	7.4	Núcleo del sistema (Pub/Sub, Streams, Hashes, Sorted Sets, TTL)
ioredis	5	Cliente de Redis con reconexión automática y pipelines
Zod	4	Validación de eventos, órdenes y archivos de configuración
Express + Socket.IO	5 / 4	API REST y comunicación en tiempo real con el navegador
React + Vite	19 / 7	Aplicación web del dashboard
Tailwind CSS · Recharts · Zustand · Lucide	4 · 3 · 5	Estilos, gráficas, estado de la interfaz e íconos
PostgreSQL + node-postgres	18 / 8	Histórico permanente y reportes SQL
Docker + Docker Compose + nginx	—	Empaquetado y ejecución reproducible; nginx sirve el dashboard
Vitest	4	Pruebas unitarias y de integración

5. Cómo funciona: el recorrido completo de un evento

La mejor forma de entender el sistema es seguir **un evento real** de principio a fin. Tomamos el evento **evt_009115**: a las 18:59:04 entró una motocicleta a la **Zona M2** (capacidad 80), que pasó de 72 a **73 vehículos**. Es exactamente el ejemplo del enunciado: $73/80 = 91,25\%$ de ocupación, estado **CRÍTICO**.

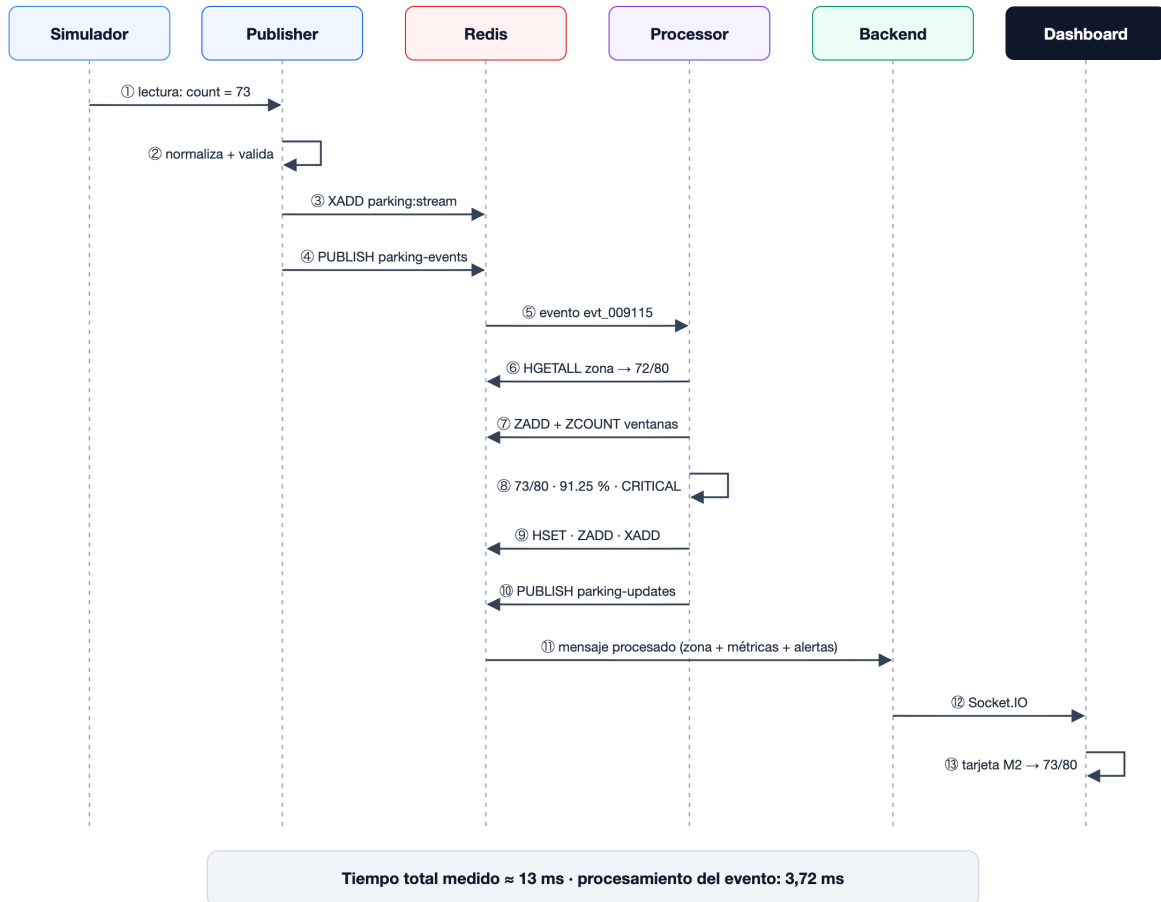


Figura 3. Diagrama de secuencia del evento evt_009115: 13 pasos desde el sensor hasta la pantalla.

Paso 1 · El sensor detecta el vehículo

Cada zona tiene un **sensor virtual** (**ZoneSimulator**). Antes de registrar la entrada, el sensor verifica que haya espacio: si la zona estuviera llena (80/80), la entrada se descarta. Como había 72, la aplica y queda en 73. El sensor entrega una lectura en **su propio formato**, como lo haría un dispositivo real:

```
{
  "sensorId": "sns-motos-02-in",
  "zoneId": "motos-02",
  "direction": "IN",
  "count": 73,
  "capacity": 80,
  "detectedAt": 1790121544758,
  "simClock": "15:05",
  "window": { "in": 7, "out": 4 },
  "scenario": "NEAR_FULL",
  "manual": false
}
```

Observe que el sensor informa el **conteo absoluto** (73) y no sólo "+1". Es una decisión de diseño: si algún componente pierde eventos, la siguiente lectura corrige el valor automáticamente.

Paso 2 · Normalización

El Publisher convierte la lectura cruda al **modelo común de eventos** que entienden todos los servicios (`EventNormalizer`):

Lectura del sensor	Evento normalizado	Qué se hace
<code>zoneId: "motos-02"</code>	<code>entity_id: "MOTOS-02"</code>	Se pasa a mayúsculas y se busca en el catálogo de zonas
<code>direction: "IN"</code>	<code>event_type: "VEHICLE_ENTERED"</code>	IN → VEHICLE_ENTERED · OUT → VEHICLE_EXITED
<code>detectedAt: 1790121544758</code>	<code>timestamp: "2026-09-22T23:59:04.758Z"</code>	Epoch en milisegundos → fecha ISO-8601
<code>count: 73</code>	<code>occupied: 73 · available: 7 · occupancy: 91.25</code>	Se calculan disponibles y porcentaje
<code>capacity: 80</code>	<code>capacity: 80</code>	La capacidad se toma del catálogo <code>config/zones.json</code> , no del sensor
—	<code>location: {latitude: null, longitude: null}</code>	No se inventan coordenadas
—	<code>metadata: {source, schema_version, generated_at, ...}</code>	Trazabilidad y marca de tiempo para medir la latencia

Paso 3 · Validación

El evento se revisa con un esquema **Zod** (`SensorEventSchema`). Si algo no cumple, el evento **no se publica** y se registra como rechazado. Se verifica, entre otras cosas:

- `event_id` con formato `evt_000001`; `event_type` dentro de la lista permitida; `timestamp` en ISO-8601.
- `occupied` entero y **nunca mayor que** `capacity` ni negativo.
- `available` **exactamente igual a** `capacity - occupied`.
- `occupancy` igual a `occupied / capacity × 100` (con tolerancia de 0,01).

- **vehicle_type** sólo **CAR** o **MOTORCYCLE**; **location** con latitud y longitud nulas o válidas.

Paso 4 · Publicación en Redis (dos mecanismos a la vez)

El Publisher obtiene un identificador único con un contador atómico de Redis (**INCR parking:seq:event** → 9115 → **evt_009115**) y luego escribe el evento en **dos lugares**:

```
XADD parking:stream MAXLEN ~ 10000 * event_id evt_009115 event_type VEHICLE_ENTERED
  entity_id MOTOS-02 timestamp 2026-09-22T23:59:04.758Z source SIMULATOR
  occupied 73 capacity 80 occupancy 91.25 payload {...JSON completo...}
→ "1790121544759-0" (ID asignado por Redis: milisegundos-secuencia)

PUBLISH parking-events {...evento con metadata.stream_id = "1790121544759-0"...}
→ 1 (un suscriptor recibió el mensaje: el Processor)
```

EN PALABRAS SIMPLES

El **Stream** es como una bitácora donde se anota el evento y queda guardado. **Pub/Sub** es como el grupo de WhatsApp: avisa al instante a quien esté conectado. Se usan los dos porque se complementan: el aviso es inmediato y, si alguien no estaba conectado, puede leer la bitácora después.

El evento que viaja, ya normalizado, validado y con su ID de Stream, es exactamente este (copiado del sistema):

```
{
  "event_id": "evt_009115",
  "event_type": "VEHICLE_ENTERED",
  "entity_id": "MOTOS-02",
  "timestamp": "2026-09-22T23:59:04.758Z",
  "location": { "latitude": null, "longitude": null },
  "data": {
    "vehicle_type": "MOTORCYCLE",
    "zone_name": "Zona M2",
    "capacity": 80,
    "occupied": 73,
    "available": 7,
    "occupancy": 91.25,
    "entries_per_minute": 7,
    "exits_per_minute": 4
  },
  "metadata": {
    "source": "SIMULATOR",
    "schema_version": "1.0",
    "generated_at": 1790121544759,
    "simulated_time": "15:05",
    "sensor_id": "SNS-MOTOS-02-IN",
    "scenario": "NEAR_FULL",
    "stream_id": "1790121544759-0"
  }
}
```

Paso 5 · El Subscriber recibe el evento

El Subscriber tiene una conexión dedicada a Redis en modo suscripción (una conexión que hace **SUBSCRIBE** sólo puede escuchar). Al recibir el mensaje lo vuelve a validar con Zod y lo pone en una **cola serial**: los eventos se procesan uno detrás de otro, en el mismo orden en que llegaron, para que nunca se mezclen dos actualizaciones de la misma zona.

Paso 6 · El Processor lee el estado anterior

```
HGETALL parking:zone:MOTOS-02
→ occupied 72 · available 8 · occupancy 90 · status CRITICAL · trend ... · total_entries ... (28 campos)
```

Con el estado anterior puede calcular variaciones y detectar si se perdió algún evento: esperaba $72 + 1 = 73$ y el sensor informa 73, así que todo cuadra. (Si no cuadrara, adopta el valor del sensor y registra una resincronización).

Paso 7 · Ventanas deslizantes (entradas por minuto)

Registra la entrada en un **Sorted Set** cuyo puntaje es el instante del evento, borra lo que tenga más de 15 minutos y cuenta cuántas entradas hubo en el último minuto, en los últimos 5 y en los últimos 15:

```
ZADD parking>window:entries:MOTOS-02 1790121544759 evt_009115
ZREMRANGEBYSCORE parking>window:entries:MOTOS-02 -inf (hace 15 min)
ZCOUNT parking>window:entries:MOTOS-02 (hace 60 s) +inf → entradas del último minuto
EXPIRE parking>window:entries:MOTOS-02 960 → la clave caduca si la zona queda inactiva
```

Paso 8 · Cálculo del nuevo estado

Variable	Cálculo	Resultado
Ocupados	lectura del sensor (entre 0 y la capacidad)	73
Disponibles	$80 - 73$	7
Ocupación	$73 / 80 \times 100$	91,25 %
Estado	91,25 está entre 90 y 99,99	CRITICAL
Variación	$91,25 - 90$ (anterior)	+1,25 pp
Tendencia	ocupación actual – ocupación de hace 30 s, comparada con ± 3 pp	RISING / STABLE / FALLING
Totales	total_entries de la zona + 1	acumulado + 1

Paso 9 · Alertas y eventos derivados

El motor de alertas compara el nuevo estado con el nivel anterior de la zona. Como M2 **ya estaba en nivel crítico**, no se crea ninguna alerta nueva (así se evitan duplicados). Si hubiera pasado de advertencia a crítico, se habría resuelto la alerta de advertencia, creado una alerta **HIGH_OCCUPANCY** y emitido el evento derivado **OCCUPANCY_CRITICAL**. El detalle está en el capítulo 10.

Paso 10 · Guardado del resultado

Todo se escribe en Redis en **un solo viaje** agrupando comandos (pipeline):

```
HSET    parking:zone:MOTOS-02 occupied 73 available 7 occupancy 91.25 status CRITICAL ...
(estado actual)
ZADD    parking:ranking:occupancy 91.25 MOTOS-02
(rankings)
HINCRBY parking:stats:entries MOTOS-02 1
(estadísticas)
XADD    parking:stream ... (sólo si hubo eventos derivados como PARKING_FULL)
(histórico)
SET     parking:processor:checkpoint 1790121544759-0
(último procesado)
```

Paso 11 · Métricas globales

Lee las seis zonas y el ranking, calcula la ocupación global, la de carros, la de motos, las zonas críticas, la zona más ocupada y las ventanas agregadas, y las guarda en **parking:metrics:global**, **parking:metrics:cars** y **parking:metrics:motorcycles**.

Paso 12 · Del Processor a la pantalla

El Processor publica en **parking-updates** un mensaje con el evento, el nuevo estado de la zona, las métricas y las alertas. El Backend, suscrito a ese canal, lo reenvía por **Socket.IO** a todos los navegadores abiertos. El dashboard, sin recargar:

- actualiza la tarjeta de la Zona M2 (73/80, barra, 7 disponibles) y le da un destello de medio segundo;
- enciende un cuadro más en la cuadrícula de espacios;
- recalcula los KPI de la parte superior (ocupación global, motos, disponibles);
- agrega la fila **VEHICLE_ENTERED Zona M2 73/80** en la actividad en tiempo real;
- pulsa el indicador **EVENT RECEIVED** y actualiza la latencia medida.

Los registros reales de los tres servicios

Así quedó el evento en los logs de cada servicio (misma hora, mismo identificador):

```

18:59:04 INFO [publisher] VEHICLE_ENTERED MOTOS-02 published event_id=evt_009115
occupied=73/80
              occupancy=91.25 stream_id=1790121544759-0 receivers=1
18:59:04 INFO [processor] Processed evt_009115 type=VEHICLE_ENTERED zone=MOTOS-02
occupied=73/80
              occupancy=91.25 status=CRITICAL trend=STABLE ms=3.72
18:59:04 INFO [backend:ws] Dashboard update emitted event_id=evt_009115 zone=MOTOS-02
              occupancy=91.25 clients=1

```

Medición de la latencia

Cada evento lleva marcas de tiempo que permiten medir cuánto tarda en llegar a la pantalla:

Marca	Quién la pone	Significado
<code>generated_at</code>	Publisher	Instante en que se generó el evento
<code>processed_at</code>	Processor	Instante en que terminó de procesarlo
<code>emitted_at</code>	Backend	Instante en que lo envió por WebSocket
<code>received_at</code>	Navegador	Instante en que llegó a la pantalla

Latencia de extremo a extremo = `received_at` – `generated_at`. El encabezado del dashboard muestra el promedio de los últimos 50 eventos: entre **9 y 13 ms** en las pruebas.

PARA LA EXPOSICIÓN

Para explicar el recorrido en 30 segundos: "El sensor detecta la moto, el Publisher arma el evento, lo valida y lo publica en Redis en dos lugares: la bitácora (Stream) y el canal de avisos (Pub/Sub). El Processor lo recibe, lee cómo estaba la zona, calcula el nuevo estado, las métricas y las alertas, guarda todo en Redis y avisa al Backend, que lo empuja por WebSocket al tablero. Todo toma unos 13 milisegundos".

En vivo: abrir el panel de simulación, presionar **Forzar VEHICLE_ENTERED** en una zona y mostrar cómo la tarjeta cambia sin recargar, mientras en la terminal aparece la línea del Publisher.

6. El simulador de sensores

Como no hay sensores físicos, el proyecto incluye un **simulador propio** que no depende de ninguna API externa. Cada zona es una entidad monitoreada con un sensor virtual en su entrada y su salida.

6.1 Por qué no se usan números aleatorios

El enunciado prohíbe generar valores independientes como `occupied = Math.floor(Math.random() * capacity)` cada 5 segundos, porque produciría saltos imposibles (de 20 % a 90 % y de vuelta a 15 %). El simulador mantiene **continuidad**: la ocupación futura depende siempre de la anterior.

```
occupied(t + 1) = occupied(t) + entradas - salidas
con la restricción: 0 ≤ occupied ≤ capacity
```

Cada movimiento cambia la ocupación en exactamente ± 1 . Una entrada en una zona llena o una salida en una zona vacía **se descartan**: el sensor no deja que ocurra lo imposible. Una prueba automatizada recorre un día completo verificando esta regla en cada lectura.

6.2 Cómo se decide cuántos vehículos entran y salen

En cada ciclo (cada `SIMULATION_INTERVAL_MS`, por defecto 5 s; 2 s en la demostración) el simulador calcula para cada zona:

1. **Cuántos movimientos habrá.** Se usa una distribución de Poisson, que modela llegadas aleatorias con una tasa promedio: $\lambda = \text{intensidad} \times \text{capacidad} \times \text{minutos simulados del ciclo} \times \text{factor de demanda de la zona}$.
2. **Si cada movimiento es entrada o salida.** Es entrada con probabilidad `entryWeight` (que depende de la hora del día) y salida en caso contrario.
3. **Cuándo ocurre dentro del ciclo.** Los movimientos se reparten a lo largo del intervalo, para que el tablero reciba eventos de forma continua y no en ráfagas.

DETALLE TÉCNICO

Ejemplo: Zona A (60 espacios) a las 07:30 (perfil de alta entrada, intensidad 0,017), ciclo de 2 s en modo acelerado (= 2 minutos simulados):

$\lambda = 0,017 \times 60 \times 2 \times 1,0 = 2,04$ movimientos esperados por ciclo. Con `entryWeight = 0,80` son en promedio 1,6 entradas y 0,4 salidas: la zona gana $\approx 1,2$ vehículos por ciclo.

EN PALABRAS SIMPLES

La distribución de Poisson describe cosas como "cuántos carros llegan a la portería en un minuto": a veces 0, a veces 1, a veces 4, pero con un promedio estable. Así los datos se ven naturales sin dejar de seguir un patrón.

6.3 Perfiles de demanda del día

Franja horaria	Perfil	Prob. entrada	Prob. salida	Intensidad	Qué representa
06:00 – 07:00	MEDIUM	0,60	0,40	0,006	Llegadas tempranas
07:00 – 08:00	HIGH_ENTRY	0,80	0,20	0,017	Llegada masiva a clases
08:00 – 12:00	STABLE	0,50	0,50	0,005	Ocupación estable en la mañana
12:00 – 14:00	HIGH_EXIT	0,25	0,75	0,0045	Salida al almuerzo
14:00 – 17:00	MEDIUM	0,55	0,45	0,020	Jornada de la tarde
17:00 – 18:00	HIGH_EXIT	0,15	0,85	0,016	Salida al final del día
18:00 – 06:00	LOW	0,30	0,70	0,006	Noche: el parqueadero se vacía

Estos valores son **parámetros de simulación** (no mediciones reales) y están en `config/simulation.json`. La figura muestra el resultado de simular un día completo en el escenario normal:

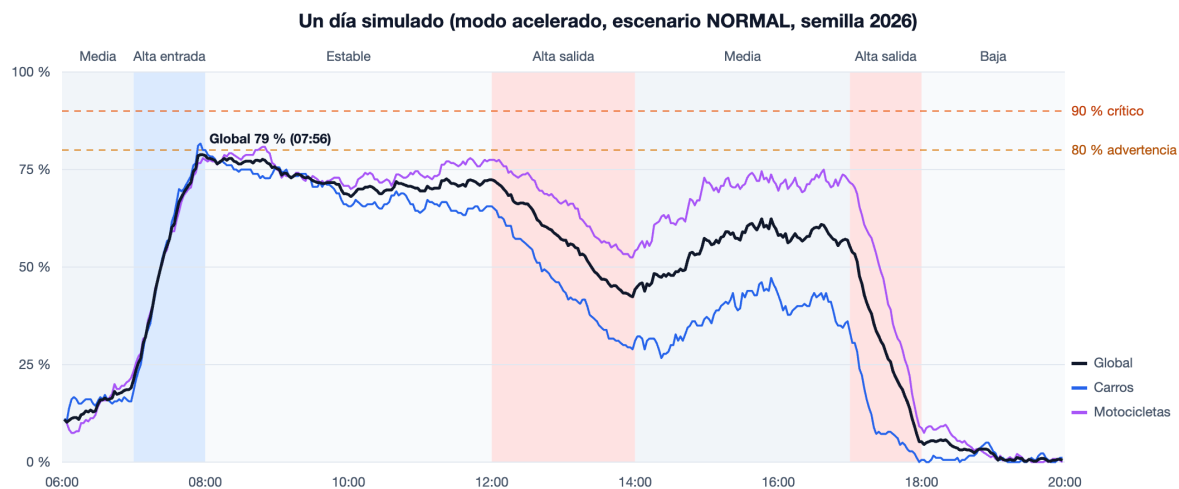


Figura 4. Un día simulado: subida rápida de 07:00 a 08:00, meseta estable, bajada al mediodía, recuperación en la tarde y vaciado desde las 17:00.

6.4 El "techo" de saturación

En la vida real, cuando una zona está casi llena los conductores buscan otra. El simulador lo imita: por encima de un techo (75 % en el escenario normal), la probabilidad de entrada se reduce de forma cuadrática: $p_{\text{entrada}} \times ((1 - r) / (1 - \text{techo}))^2$, donde r es la ocupación actual.

Con $r = 0,80$ la probabilidad de entrada en la hora pico baja de 0,80 a 0,51, casi igual a la de salida: la zona se **equilibra alrededor del 80 %**. Por eso el escenario NORMAL genera advertencias ocasionales pero nunca alertas críticas, tal como pide el enunciado. En el escenario de alta demanda el techo sube a 97 % y las zonas sí se acercan a la saturación.

6.5 Tiempo simulado: dos modos

Modo	Cómo avanza el reloj	Para qué sirve
ACCELERATED_DEMO	60 veces más rápido: 1 minuto real = 1 hora simulada . Recorre la jornada 06:00–20:00 y luego pasa al día siguiente. Inicia a las 06:30.	La exposición: en unos 14 minutos se ve un día completo (llegada masiva, estabilidad, saturación, salidas y recuperación).
REAL_TIME_PROFILE	Usa la hora actual de Bogotá.	Funcionamiento realista: a las 7:30 de la mañana real se simula la llegada masiva.

La **velocidad x1 / x2 / x5** del panel acorta el intervalo real entre ciclos (5 000 ms → 2 500 ms → 1 000 ms), acelerando aún más la simulación.

6.6 Escenarios especiales

Durante la demostración no se puede esperar a que ocurra una zona llena "por casualidad". El panel de simulación permite forzar escenarios sobre una zona o sobre todas:

Escenario	Qué hace	Cuándo termina
NORMAL	Sigue el perfil del día con techo del 75 %. Valores suaves, sin alertas críticas.	—
HIGH_DEMAND	Multiplifica la intensidad $\times 1,8$ y sube la probabilidad de entrada +0,20 (techo 97 %). Tendencia ascendente.	Al cambiar de escenario
MASS_ENTRY	Ráfaga de entradas: 30 % de la capacidad en 3 ciclos. Dispara "crecimiento inusual".	Solo, tras 3 ciclos
MASS_EXIT	Ráfaga de salidas: 35 % de la capacidad en 3 ciclos. Prueba la recuperación.	Solo, tras 3 ciclos
NEAR_FULL	Sube ≈ 6 % por ciclo hasta el 90 % y oscila entre 90 % y 99 % sin llegar a llenarse.	Al cambiar de escenario
FULL	Sube ≈ 8 % por ciclo hasta $occupied = capacity \rightarrow PARKING_FULL$. Luego, de vez en cuando sale un vehículo y otro ocupa el espacio.	Al cambiar de escenario
RECOVERY	Salidas progresivas de ≈ 6 % por ciclo (100 % $\rightarrow$ 94 % $\rightarrow$ 88 % $\rightarrow$ 82 % ...) hasta el 70 %.	Solo, al llegar al 70 %

Además se pueden **forzar eventos manuales** (1, 5 o 10 entradas o salidas), **pausar**, **reanudar**, **cambiar la velocidad y reiniciar el día**. Cada cambio de escenario se publica como evento **SCENARIO_CHANGED** en el canal **system-events**.

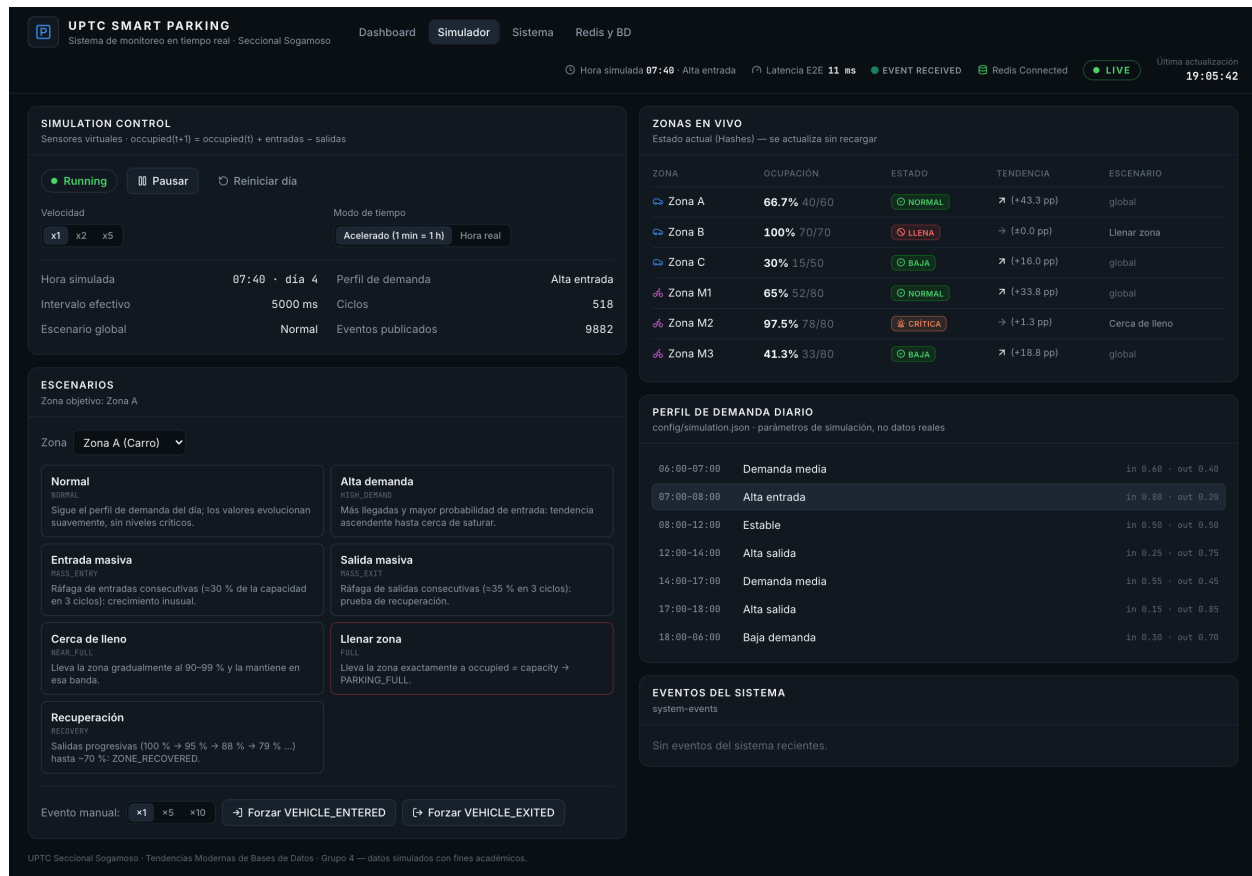


Figura 5. Panel de control del simulador (/simulator): estado, velocidad, modo de tiempo, escenarios, eventos manuales, zonas en vivo y el perfil de demanda activo resaltado.

6.7 Cómo llegan las órdenes al simulador

1. El usuario presiona un botón en /simulator (por ejemplo "Llenar zona" en Zona C).
2. El navegador hace `POST /api/simulator/commands` con `{"action": "SET_SCENARIO", "scenario": "FULL", "zone_id": "CARS-C"}`.
3. El Backend valida la orden con Zod y la publica en el canal `simulator-commands`.
4. El Publisher, suscrito a ese canal, la valida de nuevo, la aplica y publica `SCENARIO_CHANGED`.
5. En los ciclos siguientes la Zona C se llena y el Processor genera `PARKING_FULL` y la alerta correspondiente.

6.8 Reproducibilidad y continuidad ante reinicios

- **Semilla fija (SIMULATION_SEED):** el generador pseudoaleatorio (mulberry32) produce exactamente la misma secuencia con la misma semilla. Una prueba automatizada lo verifica.
- **Estado guardado en Redis:** el Publisher guarda en `parking:simulator:state` el reloj, el día, el escenario y la velocidad. Si se reinicia, continúa donde iba (restaura también la ocupación de cada zona) en lugar de empezar desde cero.

PARA LA EXPOSICIÓN

Pregunta típica: "¿Cómo generan los eventos?". Respuesta: "Con un simulador que conserva el estado anterior de cada zona y genera entradas o salidas según perfiles de demanda por hora, con llegadas tipo Poisson. No son números aleatorios independientes: siempre respetan la capacidad y la continuidad".

7. Redis en detalle

7.1 Por qué Redis

Redis resulta adecuado para la actividad porque almacena la información directamente en memoria y ofrece mecanismos orientados a eventos. En nuestra arquitectura Redis no se utiliza únicamente como almacenamiento, sino también como mecanismo de comunicación mediante Pub/Sub, como histórico reciente mediante Streams, como almacenamiento del estado actual mediante Hashes y como mecanismo de ranking mediante Sorted Sets.

Ventajas que aprovecha el proyecto: baja latencia (≈ 4 ms para procesar un evento con unos 6 viajes a Redis), lecturas y escrituras muy rápidas, estructuras especializadas, mensajería integrada y expiración automática de datos.

7.2 Convención de nombres de claves

Todas las claves siguen el patrón `parking:<catagoría>:<identificador>`, por ejemplo `parking:zone:MOTOS-02` o `parking:metrics:global`. Están definidas en un solo archivo (`packages/shared/src/constants/redis-keys.ts`) que usan todos los servicios.

7.3 Pub/Sub — comunicación inmediata

Qué es: un sistema de mensajería. Un servicio publica un mensaje en un canal y Redis se lo entrega al instante a todos los que estén suscritos a ese canal en ese momento.

EN PALABRAS SIMPLES

Es como una emisora de radio: el locutor habla y todos los que tienen el radio encendido lo escuchan en ese instante. Quien tenía el radio apagado **no puede escuchar después** lo que se dijo.

```
PUBLISH parking-events '{"event_id":"evt_009115","event_type":"VEHICLE_ENTERED",...}' → 1
receptor
SUBSCRIBE parking-events →
recibe en vivo
```

El proyecto usa cinco canales (ver 4.2). El Publisher registra cuántos suscriptores recibieron cada mensaje: si el Processor está caído aparece `receivers=0` con la advertencia de que Pub/Sub no conserva mensajes.

7.4 Streams — histórico reciente

Qué es: un registro ordenado donde cada entrada se agrega al final con un ID basado en el tiempo (`milisegundos-secuencia`). Las entradas **permanecen** y se pueden leer después, por rango de IDs o de tiempo.

EN PALABRAS SIMPLES

Es como la bitácora de la portería: cada entrada y salida queda anotada con la hora. Cualquiera puede leerla después, aunque no estuviera presente cuando se escribió.

Stream	Contenido	Límite
<code>parking:stream</code>	Todos los eventos: de sensor, derivados (PARKING_FULL, ZONE_RECOVERED...) y del sistema	MAXLEN ~ 10 000
<code>parking:timeseries</code>	Una muestra de métricas cada 2 s (alimenta las gráficas)	MAXLEN ~ 5 000 (≈ 2,7 h)
<code>parking:alerts:stream</code>	Historial de alertas levantadas y resueltas	MAXLEN ~ 2 000

```
XADD parking:stream MAXLEN ~ 10000 * event_id evt_009115 event_type VEHICLE_ENTERED ...
XRANGE parking:stream - +           → todos los eventos guardados, del más antiguo al
más nuevo
XREVRANGE parking:stream + - COUNT 10 → los 10 más recientes
XRANGE parking:timeseries 1790120644759 + → las muestras desde un instante (los IDs son
tiempo)
```

Los Streams permiten además **consumer groups**: varios lectores que se reparten las entradas y confirman cada una con **XACK**. El servicio archiver lo usa para copiar el histórico a PostgreSQL sin perder nada (capítulo 13).

7.5 Pub/Sub vs Streams: por qué se usan los dos

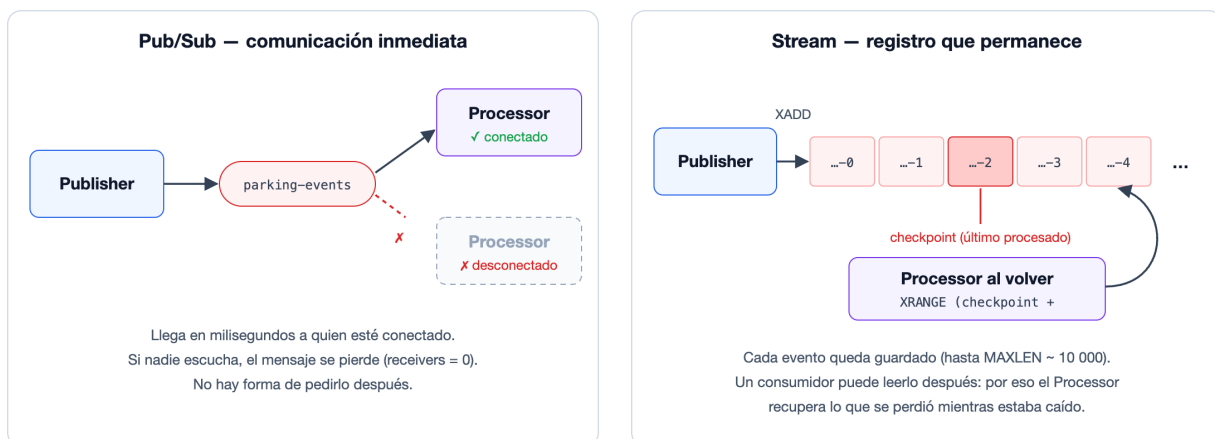


Figura 6. Pub/Sub entrega al instante pero no guarda; el Stream guarda y permite recuperar lo que se perdió.

	Pub/Sub	Streams
Objetivo	Comunicación inmediata	Registro que permanece
Suscriptor desconectado	Pierde los mensajes	Los lee al volver
Lectura del pasado	No es posible	Sí, por rango de IDs o de tiempo

	Pub/Sub	Streams
Reparto entre consumidores	No	Sí, con consumer groups y confirmación
Uso en el proyecto	Avisar en vivo al Processor y al Backend	Histórico, recuperación del Processor, gráficas, archivo en PostgreSQL

IDEA CLAVE

Pub/Sub da **inmediatez**; el Stream da **durabilidad**. Juntos permiten un sistema en tiempo real que no pierde eventos aunque un consumidor se caiga.

7.6 Hashes — estado actual

Qué es: un objeto con varios campos bajo una sola clave, como una ficha. Se lee completo con **HGETALL** o se actualizan sólo algunos campos con **HSET**.

EN PALABRAS SIMPLES

Es la ficha de cada zona pegada en la pared: dice cuántos vehículos hay **ahora**. Cuando cambia algo se borra el número viejo y se escribe el nuevo.

Cada zona tiene su Hash **parking:zone:<id>** con 28 campos:

Campo	Significado
zone_id · zone_name · vehicle_type · capacity	Identificación de la zona (desde config/zones.json)
occupied · available · occupancy · status	Ocupados, disponibles, porcentaje y categoría (LOW/NORMAL/WARNING/CRITICAL/FULL)
previous_occupancy · occupancy_delta · previous_status	Valores del evento anterior y variación
entries_per_minute · exits_per_minute	Entradas y salidas en el último minuto
entries_5m · exits_5m · entries_15m · exits_15m	Entradas y salidas en 5 y 15 minutos
total_entries · total_exits	Acumulados desde el inicio
trend · trend_delta	Tendencia (RISING/FALLING/STABLE) y su variación en 30 s
avg_occupancy_5m · variation_5m	Promedio y variación de los últimos 5 minutos
last_event · last_event_id · timestamp · last_update · simulated_time	Trazabilidad: último evento, instantes y hora simulada

También son Hashes las métricas agregadas (**parking:metrics:global**, **:cars**, **:motorcycles**, 22 campos cada uno), las alertas activas (**parking:alerts:active**), el nivel de alerta de cada zona, las estadísticas y el estado del simulador.

7.7 Estado actual vs histórico reciente

Esta diferencia es central en el enunciado. El **estado actual** es la última información conocida de cada zona; el **histórico reciente** es la secuencia de eventos que llevó a ese estado y permite construir las gráficas.



Figura 7. La misma Zona M2 vista de dos formas: la ficha actual (Hash) y la bitácora de eventos (Stream). Datos reales del 22 de septiembre.

	Estado actual	Histórico reciente
Estructura	Hash <code>parking:zone:MOTOS-02</code>	Stream <code>parking:stream</code> (y <code>parking:timeseries</code>)
Pregunta que responde	¿Cómo está M2 ahora ?	¿Cómo evolucionó M2?
Ejemplo	73 ocupados · 7 disponibles · 91,25 % · CRITICAL	18:58 → 93,75 % · 92,5 % · 91,25 % · 90 % · 91,25 % ...
Comando	<code>HGETALL parking:zone:MOTOS-02</code>	<code>XRANGE parking:stream - +</code>
Se usa para	Tarjetas, KPIs, alertas	Gráficas, detalle de la zona, recuperación, archivo

7.8 Sorted Sets — ranking y ventanas de tiempo

Qué es: un conjunto donde cada elemento tiene un puntaje numérico y Redis lo mantiene **siempre ordenado** por ese puntaje.

Ranking de ocupación

Cada vez que cambia una zona se actualiza su puntaje con su porcentaje de ocupación. Consultar la zona más ocupada es inmediato:

```
ZADD parking:ranking:occupancy 91.25 MOTOS-02
ZREVRANGE parking:ranking:occupancy 0 -1 WITHSCORES
1) CARS-B      100      ← zona más ocupada (ZREVRANGE ... 0 0)
2) MOTOS-02   90
3) CARS-C      60
4) MOTOS-03   37.5
5) CARS-A     36.67
6) MOTOS-01   36.25
```

Ventanas deslizantes (entradas y salidas por minuto)

Cada entrada se guarda en `parking:window:entries:<zona>` con el instante del evento como puntaje. Para saber cuántas entradas hubo en el último minuto basta contar los elementos con puntaje mayor que "ahora - 60 s" (`ZCOUNT`). Lo que tiene más de 15 minutos se borra (`ZREMRANGEBYSCORE`) y la clave tiene un TTL de 16 minutos. Ejemplo real:

```
ZRANGE parking:window:entries:MOTOS-02 -3 -1 WITHSCORES
evt_008835 1790121487241
evt_009005 1790121522252
evt_009115 1790121544759 ← la entrada del ejemplo del capítulo 5
TTL parking:window:entries:MOTOS-02 → 950 segundos
```

7.9 TTL — información que caduca sola

Qué es: un tiempo de vida en segundos. Cuando se cumple, Redis borra la clave automáticamente.

Clave	TTL	Para qué
<code>parking:heartbeat:<servicio></code>	10 s (renovado cada ≈ 3 s)	Saber si un servicio está vivo: si deja de renovarla, expira y aparece OFFLINE
<code>parking:alert:cooldown:<zona>:<tipo></code>	30 s	No repetir una alerta justo después de que la zona se recuperó
<code>parking:temporary:metric:*</code>	10 s	Eventos por segundo y tiempo de procesamiento instantáneos
<code>parking:window:*</code>	960 s	Las ventanas de zonas inactivas desaparecen solas

EN PALABRAS SIMPLES

El heartbeat funciona como un "estoy vivo" que cada servicio repite cada 3 segundos y que vale 10 segundos. Si un servicio se cae deja de repetirlo, la clave se vence sola y el panel de sistema lo muestra OFFLINE. Nadie tuvo que avisar que se cayó.

7.10 Mapa completo de claves

Estructura	Clave	Escribe	Lee
Hash	<code>parking:zone:<id></code>	Processor	Backend, Publisher, Archiver
Hash	<code>parking:metrics:{global,cars,motorcycles}</code>	Processor	Backend
Hash	<code>parking:alerts:active · parking:alert:state:<id></code>	Processor	Backend · Processor
Hash	<code>parking:stats:{entries,exits,system}</code>	Processor, Publisher	Backend
Hash	<code>parking:simulator:state</code>	Publisher	Backend, Publisher
Stream	<code>parking:stream</code>	Publisher, Processor	Processor, Backend, Archiver
Stream	<code>parking:timeseries</code>	Processor	Processor, Backend
Stream	<code>parking:alerts:stream</code>	Processor	Backend, Archiver
Sorted Set	<code>parking:ranking:occupancy</code>	Processor	Processor, Backend
Sorted Set	<code>parking>window:{entries,exits}:<id></code>	Processor	Processor
String + TTL	<code>parking:heartbeat:* · parking:alert:cooldown:* · parking:temporary:metric:*</code>	Todos · Processor	Backend · Processor
String	<code>parking:seq:event · parking:seq:alert · parking:processor:checkpoint</code>	Publisher, Processor	Processor

7.11 Control del crecimiento

Dato	Estrategia
Streams	<code>XADD ... MAXLEN ~ N</code> : Redis recorta automáticamente las entradas más antiguas (10 000 eventos, 5 000 muestras, 2 000 alertas)
Ventanas	<code>ZREMRANGEBYSCORE</code> a 15 minutos + <code>EXPIRE 960</code>
Datos temporales	TTL: heartbeats, cooldowns y métricas instantáneas desaparecen solos
Alertas resueltas	Se eliminan del Hash de activas (<code>HDEL</code>); quedan en el Stream limitado y en PostgreSQL
Memoria	<code>maxmemory 256mb</code> con política <code>volatile-lru</code> : si faltara memoria sólo se borrarían claves con TTL, nunca el estado actual
Histórico de largo plazo	Se traslada a PostgreSQL, que tiene su propia retención (30 días por defecto)

Medición real: con el Stream en su límite de 10 000 eventos, Redis ocupa **≈ 9 MB** de memoria y 45 claves, muy lejos del límite de 256 MB.

7.12 Persistencia y reinicios

Redis está configurado con instantáneas **RDB** (`save 300 100` y `save 60 10000`) guardadas en un volumen de Docker: al reiniciar el contenedor, el estado se recupera. En producción podría activarse **AOF**

(**appendonly yes**) para perder como máximo un segundo de datos. Aun sin persistencia, el sistema se reconstruye solo porque los sensores informan conteos absolutos.

8. El Subscriber y el Processor

8.1 El Subscriber

- Usa una **conexión dedicada** a Redis: una conexión en modo **SUBSCRIBE** no puede ejecutar otros comandos.
- Valida cada mensaje con el esquema Zod y descarta los inválidos (quedan contados en **events_invalid**).
- Entrega los eventos a una **cola serial**: se procesan uno a la vez, en orden de llegada.
- Al iniciar y cada vez que se reconecta, **recupera desde el Stream** los eventos que se perdió (sección 8.3).

8.2 Los pasos del Processor para cada evento

1. **Descarta duplicados**: si el **stream_id** del evento ya fue procesado, lo ignora.
2. **Lee el estado anterior** de la zona: **HGETALL parking:zone:<id>**.
3. **Registra el movimiento en la ventana** y cuenta entradas/salidas de 1, 5 y 15 minutos (**ZADD**, **ZCOUNT**, **ZREMRANGEBYSCORE**, **EXPIRE**).
4. **Calcula el nuevo estado**: ocupados, disponibles, porcentaje, categoría, variación, tendencia, promedio y variación de 5 minutos, totales.
5. **Evalúa las alertas** de la zona con la máquina de estados (capítulo 10).
6. **Deriva eventos**: **ZONE_STATUS_CHANGED**, **PARKING_FULL**, **PARKING_AVAILABLE**, **OCCUPANCY_WARNING**, **OCCUPANCY_CRITICAL**, **ZONE_RECOVERED**.
7. **Guarda** en un pipeline: **HSET** del estado, **ZADD** del ranking, **HINCRBY** de estadísticas, **XADD** de los eventos derivados y **SET** del checkpoint.
8. **Calcula las métricas globales** (lee las 6 zonas y el ranking), evalúa las alertas por tipo de parqueadero y guarda **parking:metrics:***.
9. **Publica** el resultado en **parking-updates** (y cada alerta en **parking-alerts**).

8.3 Recuperación desde el Stream (tolerancia a fallos)

Después de procesar cada evento, el Processor guarda su **checkpoint**: el ID del último evento procesado (**parking:processor:checkpoint**). Si se detiene, los eventos publicados por Pub/Sub durante ese tiempo se pierden para él, pero siguen en el Stream. Al volver:

1. Lee su checkpoint.
2. Ejecuta **XRANGE parking:stream (<checkpoint> + COUNT 250** en bloques y procesa los eventos de sensor que faltan.
3. Descarta por **stream_id** los mensajes que llegan a la vez por Pub/Sub y ya fueron procesados.
4. Publica un mensaje **RESYNC** y los dashboards recargan su estado.

Resultado real al detener el Processor 10 segundos en Docker:

```
[publisher] WARN No Pub/Sub subscriber received the event – Pub/Sub does not retain messages;  
                it remains in parking:stream  
[processor] INFO Processor checkpoint last_stream_id=1790118901723-0  
[processor] INFO Recovered 106 events missed while offline from parking:stream reason=startup  
[backend]   INFO Resync broadcast to dashboards recovered=106
```

8.4 Calibración y resincronización

Como el sensor envía el **conteo absoluto**, el Processor puede detectar inconsistencias: si esperaba $72 + 1 = 73$ y el sensor dice 75, registra una resincronización y adopta 75. Cuando una zona recibe su primera lectura (tras un arranque limpio, su Hash inicial vale 0) se trata como **calibración** y no se interpreta como un salto anormal.

8.5 El muestreador (cada 2 segundos)

Además de reaccionar a los eventos, el Processor ejecuta cada 2 segundos una tarea que: recalcula las ventanas de **todas** las zonas (aunque no hayan tenido eventos), actualiza tendencia y promedios, agrega una muestra a `parking:timeseries` (de donde salen las gráficas), guarda eventos por segundo y tiempo de procesamiento con TTL de 10 s, revisa las alertas que dependen del paso del tiempo (crecimiento inusual, cooldowns vencidos) y publica la muestra al dashboard.

9. Métricas

El enunciado pide no mostrar simplemente lo que entrega el simulador, sino **calcular información nueva**. Estas son las métricas del sistema:

#	Métrica	Cómo se calcula	Ejemplo
1	Ocupación por zona	<code>occupied / capacity × 100</code>	73 / 80 = 91,25 %
2	Espacios disponibles	<code>capacity - occupied</code>	80 - 73 = 7
3	Ocupación global	Σ ocupados / Σ capacidad × 100	246 / 420 = 58,6 %
4	Ocupación de carros	Igual, sólo zonas CAR	108 / 180 = 60 %
5	Ocupación de motos	Igual, sólo zonas MOTORCYCLE	138 / 240 = 57,5 %
6	Entradas por minuto	<code>ZCOUNT</code> de la ventana de 60 s	129 entradas / min (global)
7	Salidas por minuto	Igual para salidas	52 salidas / min
8	Zona más ocupada	<code>ZREVRANGE parking:ranking:occupancy 0 0</code>	Zona B (100 %)
9	Zonas críticas	Zonas en CRITICAL o FULL	2 de 6 (Zona B y Zona M2)
10	Variación	Ocupación actual - anterior; y - hace 5 min	+1,25 pp por evento · -10,7 pp global en 5 min
11	Tendencia	Variación en 30 s comparada con ±3 pp	RISING / FALLING / STABLE
—	Promedios por ventana	Media de las muestras de 1, 5 y 15 min	Zona M2, promedio 5 min: 92,82 %
—	Métricas internas	Eventos/s, ms por evento, latencia E2E, lag del archivo	5,5 ev/s · 4 ms · 9–13 ms

Los ejemplos provienen de la Figura 1 y del panel de detalle de la Zona M2 (Figura 13), capturados del sistema en ejecución. Las métricas por zona se guardan en su Hash; las agregadas en `parking:metrics:global`, `parking:metrics:cars` y `parking:metrics:motorcycles`.

9.1 Tendencia

Se compara la ocupación actual con la de hace 30 segundos (tomada de la serie temporal). Si la diferencia es de al menos +3 puntos porcentuales la tendencia es **RISING**; si es de -3 o menos, **FALLING**; en otro caso **STABLE**. El umbral y la ventana son configurables en `config/thresholds.json`.

```
const delta = ocupacionActual - ocupacionHace30s;
if (delta >= 3)      trend = 'RISING';
else if (delta <= -3) trend = 'FALLING';
else                trend = 'STABLE';
```

9.2 Estados de una zona

Estado	Rango de ocupación	Color e ícono en el dashboard
LOW	0 % – 59,99 %	Verde · "Baja"
NORMAL	60 % – 79,99 %	Verde · "Normal"
WARNING	80 % – 89,99 %	Amarillo · triángulo · "Advertencia"
CRITICAL	90 % – 99,99 %	Naranja-rojo · sirena · "Crítica"
FULL	100 %	Rojo · prohibido · "Llena"

Los umbrales están en `config/thresholds.json` y pueden sobrescribirse con las variables `WARNING_OCCUPANCY` y `CRITICAL_OCCUPANCY`.

10. Alertas

Una alerta no es un `alert("Zona llena")`: es un objeto con ciclo de vida (se levanta, puede escalar y se resuelve), que se guarda, se difunde y se archiva.

10.1 Modelo de una alerta

Alerta real del sistema:

```
{
  "id": "alert_000047",
  "zone_id": "MOTOS-02",
  "zone_name": "Zona M2",
  "type": "HIGH_OCCUPANCY",
  "severity": "CRITICAL",
  "message": "La Zona M2 alcanzó 90% de ocupación.",
  "value": 90,
  "threshold": 90,
  "unit": "%",
  "status": "ACTIVE",
  "timestamp": "2026-09-22T23:58:05.410Z"
}
```

Al resolverse se agregan `status: "RESOLVED"`, `resolved_at`, `resolution` (RECOVERED, ESCALATED, DEESCALATED o CONDITION_CLEARED) y un mensaje de resolución.

10.2 Tipos de alerta

Tipo	Condición	Severidad	Se resuelve cuando
OCCUPANCY_WARNING	Ocupación ≥ 80 %	WARNING	Escala a crítica o baja de 78 %
HIGH_OCCUPANCY	Ocupación ≥ 90 %	CRITICAL	Escala a llena o baja de 88 %
PARKING_FULL	occupied = capacity	CRITICAL	Baja de 98 %
LOW_AVAILABILITY	Disponibles ≤ 5	WARNING	Disponibles > 7 , o la zona se llena
UNUSUAL_OCCUPANCY_INCREASE	Sube ≥ 20 pp en 6 s	WARNING	El aumento baja de 10 pp
CAR_PARKING_FULL	Las 3 zonas de carros llenas	CRITICAL	Alguna zona libera espacio
MOTORCYCLE_PARKING_FULL	Las 3 zonas de motos llenas	CRITICAL	Alguna zona libera espacio

10.3 La máquina de estados

Cada zona tiene un **nivel de alerta** guardado en Redis (`parking:alert:state:<zona>`): NONE, WARNING, CRITICAL o FULL. Las alertas sólo se crean cuando el nivel **cambia**:

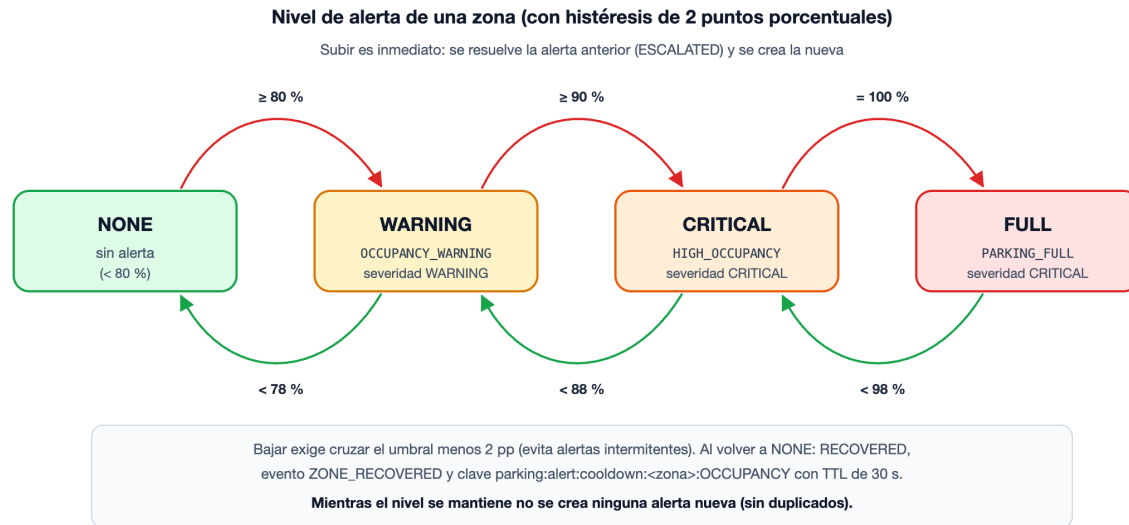


Figura 8. Máquina de estados de las alertas de ocupación. Subir es inmediato; bajar exige cruzar el umbral menos 2 puntos (histéresis).

10.4 Ejemplo completo (la prueba de la sección 131 del enunciado)

Ocupación	Nivel	Qué hace el sistema	Eventos emitidos
78 % (62/80)	NONE	Nada	—
82,5 % (66/80)	WARNING	Crea alerta OCCUPANCY_WARNING	ZONE_STATUS_CHANGED, OCCUPANCY_WARNING
91,25 % (73/80)	CRITICAL	Resuelve la advertencia (ESCALATED) y crea HIGH_OCCUPANCY	ZONE_STATUS_CHANGED, OCCUPANCY_CRITICAL
92,5 %, 91,25 %...	CRITICAL	Nada: el nivel no cambió (sin duplicados)	—
100 % (80/80)	FULL	Resuelve la crítica (ESCALATED) y crea PARKING_FULL	ZONE_STATUS_CHANGED, PARKING_FULL
75 % (60/80)	NONE	Resuelve PARKING_FULL (RECOVERED) y activa un cooldown de 30 s	PARKING_AVAILABLE, ZONE_RECOVERED

Esta secuencia está cubierta por una prueba automatizada que corre contra un Redis real y verifica cada alerta, los eventos derivados y el TTL del cooldown.

10.5 Histéresis y cooldown: cómo se evitan alertas repetidas

En una zona de 50 espacios cada vehículo equivale a 2 puntos porcentuales. Si la zona oscila entre 39 y 40 vehículos (78 % y 80 %), un sistema ingenuo crearía y cerraría una advertencia con cada auto. Se usan dos mecanismos:

- **Histéresis:** para salir de un nivel hay que bajar del umbral menos 2 pp. Una advertencia creada al 80 % sólo se resuelve cuando la ocupación baja **de 78 %**.
- **Cooldown con TTL:** al recuperarse la zona se crea `parking:alert:cooldown:<zona>:OCCUPANCY` con `EX 30`. Si en esos 30 segundos vuelve a

cruzar el 80 %, la advertencia se aplaza hasta que la clave expire. Si llega a crítica, la alerta sí se crea de inmediato (escalar nunca se bloquea).

EN PALABRAS SIMPLES

Es como un termostato: si se prende a 20 °C, no se apaga a 20,1 °C sino a 22 °C. Así no se prende y apaga cada segundo.

10.6 Dónde viven las alertas

Lugar	Qué guarda
Hash <code>parking:alerts:active</code>	Las alertas activas (campo <code><zona>:<tipo></code>); se borra el campo al resolver
Hash <code>parking:alert:state:<zona></code>	El nivel actual de la máquina de estados
String con TTL	Los cooldowns vigentes
Stream <code>parking:alerts:stream</code>	El historial RAISED / RESOLVED (MAXLEN ~ 2 000)
Canal <code>parking-alerts</code>	El aviso en tiempo real al Backend y de ahí al navegador
Tabla <code>alerts</code> (PostgreSQL)	El historial permanente con duración y motivo de resolución



Figura 9. Detalle de una alerta en el dashboard: zona, tipo, severidad, valor, umbral, fecha y estado.

PARA LA EXPOSICIÓN

Pregunta típica: "¿Cómo generan alertas?". Respuesta: "El Processor compara las métricas con umbrales configurables usando una máquina de estados por zona. Sólo crea una alerta cuando el nivel cambia, usa histéresis y un cooldown con TTL para no repetirla, y cuando la zona se recupera la resuelve y emite ZONE_RECOVERED".

11. Backend: API REST y tiempo real

El Backend (Express 5 + Socket.IO) es el puente entre Redis y el navegador. No calcula métricas: las **lee** de Redis y las **difunde**. Tiene dos conexiones a Redis: una para consultas (que falla rápido si Redis no está, para responder con un error claro) y otra dedicada a escuchar canales.

11.1 Endpoints REST

Método y ruta	Devuelve	Estructura Redis que consulta
GET /api/health	Estado general (healthy / degraded / unhealthy)	Heartbeats
GET /api/zones	Estado actual de las 6 zonas	Hashes <code>parking:zone:*</code>
GET /api/zones/:id	Estado actual de una zona	Hash de la zona
GET /api/zones/:id/history	Últimos eventos y curva de ocupación de la zona	<code>parking:stream + parking:timeseries</code>
GET /api/metrics	Métricas global, carros y motos	Hashes <code>parking:metrics:*</code>
GET /api/metrics/cars · /motorcycles	Métricas por tipo de vehículo	Hash correspondiente
GET /api/metrics/history	Muestras de los últimos N minutos	<code>XRANGE parking:timeseries</code>
GET /api/ranking	Zonas ordenadas por ocupación	<code>ZREVRANGE parking:ranking:occupancy</code>
GET /api/events	Últimos eventos (filtros por zona y tipo)	<code>XREVRANGE parking:stream</code>
GET /api/alerts · /alerts/active	Historial de alertas · alertas activas	Stream y Hash de alertas
GET /api/system/status	Estado de cada servicio, Redis, Streams, PostgreSQL	Heartbeats, INFO, XINFO
GET /api/simulator · POST /api/simulator/commands	Estado del simulador · enviar una orden	<code>parking:simulator:state · PUBLISH</code>
GET /api/debug/keys · /inspect · /pubsub	Explorador de claves (sólo lectura)	SCAN, TYPE, TTL, PUBSUB NUMSUB
GET /api/archive/stats · /daily · /events · /alerts	Histórico permanente	PostgreSQL

Todas las respuestas usan el formato `{ "data": ... }` y los errores `{ "error": { "code", "message" } }`. Si Redis está caído la API responde **503 REDIS_UNAVAILABLE** en lugar de quedarse esperando; los parámetros inválidos responden **400** con el motivo exacto.

11.2 Eventos Socket.IO

Evento	Cuándo se emite	Contenido
<code>parking:event</code>	Por cada evento (sensor, derivado o del sistema)	El evento + <code>emitted_at</code>
<code>parking:update</code>	Por cada evento de sensor procesado	Nuevo estado de la zona
<code>parking:metrics</code>	Por evento y cada 2 s	Métricas globales y, cada 2 s, una muestra para las gráficas
<code>parking:alert</code>	Cuando se levanta una alerta	La alerta
<code>parking:recovery</code>	Cuando se resuelve una alerta	La alerta resuelta
<code>parking:resync</code>	Cuando el Processor recupera eventos del Stream	Cantidad recuperada: el navegador recarga su estado
<code>system:status</code>	Cada 2 s	Estado de todos los servicios
<code>simulator:state</code>	Cuando cambia	Hora simulada, escenario, velocidad...

11.3 Estado del sistema y heartbeats

Cada servicio escribe cada ≈ 3 s su heartbeat `parking:heartbeat:<servicio>` con TTL de 10 s y algunos datos (eventos procesados, cola pendiente, estado de PostgreSQL...). El Backend los lee junto con `INFO` de Redis, las longitudes de los Streams y el `*lag*` del consumer group, y arma el estado que muestra la página `/system`.

11.4 Seguridad del panel de simulación

Si se define la variable `DEMO_CONTROL_TOKEN`, las órdenes al simulador exigen la cabecera `x-demo-token`. El explorador de Redis de `/debug` es de **sólo lectura** y sólo permite claves que empiezan por `parking:.`

12. El dashboard

La aplicación web (React) permite comprender todo el sistema en una sola vista. Tiene cuatro páginas: **Dashboard** (/), **Simulador** (/simulador), **Sistema** (/system) y **Redis y BD** (/debug).

12.1 Encabezado



Figura 10. Encabezado: navegación, hora simulada y perfil, latencia E2E, indicador de nuevo evento, conexión con Redis, estado LIVE y última actualización.

Indicador	Qué significa
Hora simulada · perfil	Reloj del simulador y perfil de demanda vigente (p. ej. 13:55 · Alta salida)
Latencia E2E	Promedio de received_at – generated_at de los últimos 50 eventos
● EVENT RECEIVED	Pulsa durante ≈ 450 ms cada vez que llega un evento: evidencia visual del tiempo real
Redis Connected	Se vuelve "Redis reconectando" si Redis se cae
● LIVE	Conexión WebSocket activa; cambia a ● RECONNECTING si se pierde
Última actualización	Hora del último dato recibido

12.2 Indicadores principales (KPI)

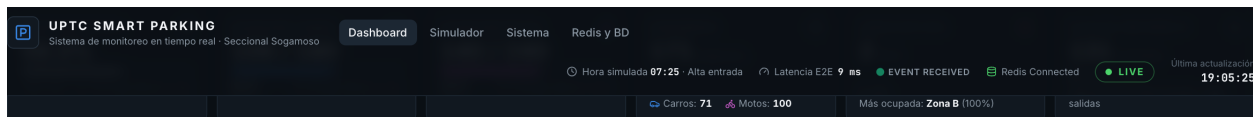


Figura 11. Fila de KPIs: ocupación global con variación de 5 minutos, carros, motos, disponibles, zonas críticas y flujo del último minuto.

12.3 Plano conceptual y tarjetas de zona

El plano muestra los dos parqueaderos con sus tres zonas cada uno. Cada tarjeta incluye: nombre y tipo, estado (ícono + texto), porcentaje, ocupados/capacidad, barra, una **cuadrícula con un cuadro por espacio** (los ocupados se rellenan), espacios disponibles, tendencia, entradas y salidas por minuto y hace cuánto se actualizó. Al recibir un cambio la tarjeta destella medio segundo. El color del borde refleja el estado (verde, amarillo, naranja, rojo).

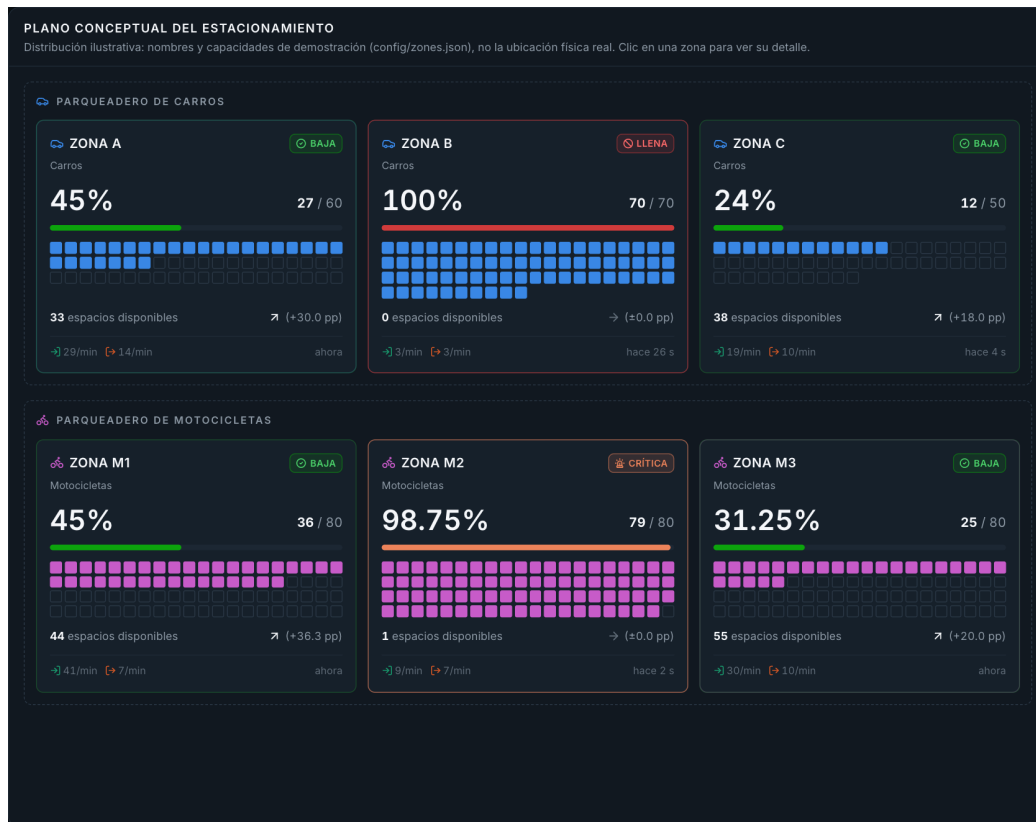


Figura 12. Plano conceptual: Zona B llena (borde rojo, 0 disponibles) y Zona M2 crítica (98,75 %). La distribución es ilustrativa, no la ubicación física real.

12.4 Detalle de una zona

Al hacer clic en una tarjeta se abre un panel lateral que muestra, lado a lado, el **estado actual** (con el comando `HGETALL` que lo produce), la **curva de ocupación de los últimos 15 minutos** (de `parking:timeseries`), las alertas activas, los **últimos eventos** (de `parking:stream`) y botones de demostración: forzar entrada, forzar salida, llenar zona y recuperación.



Figura 13. Panel de detalle de la Zona M2: estado actual del Hash y curva temporal del Stream.

12.5 Gráficas

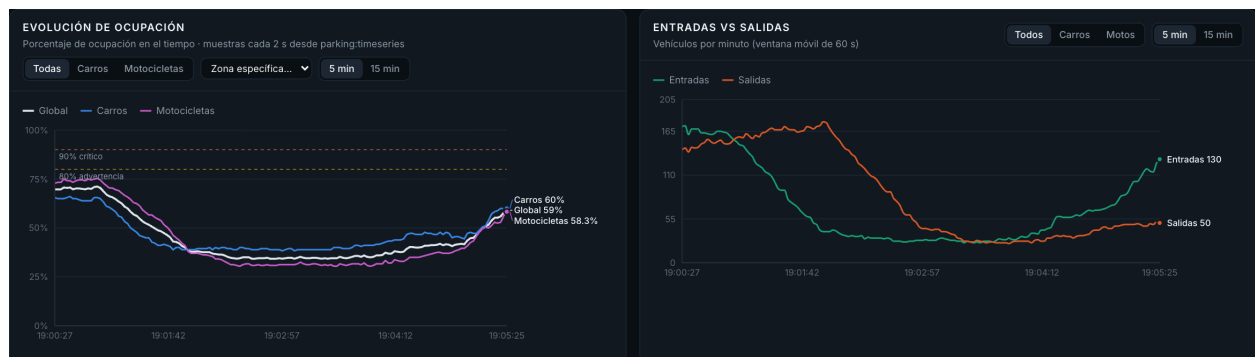


Figura 14. Gráfica 1: evolución de ocupación (filtros Todas / Carros / Motos / zona; rango 5 o 15 min; líneas de umbral 80 % y 90 %). Gráfica 2: entradas vs salidas por minuto.



Figura 15. Gráfica 3: ocupación por zona ordenada de mayor a menor (Sorted Set). A la derecha, el recorrido de un evento con contadores en vivo de cada etapa.

Gráfica	Eje X	Eje Y	Fuente
Evolución de ocupación	Tiempo	% de ocupación	parking:timeseries (muestra cada 2 s)
Entradas vs salidas	Tiempo	Vehículos por minuto	Ventanas deslizantes (Sorted Sets)
Ocupación por zona	% de ocupación	Zona	parking:ranking:occupancy

Todas las gráficas tienen leyenda, etiqueta directa al final de cada línea y cuadro de información al pasar el mouse. Los colores se verificaron con un validador de daltonismo: carros en azul, motos en violeta, entradas en verde azulado y salidas en naranja; los estados nunca dependen sólo del color (siempre llevan ícono y texto).

12.6 Actividad en tiempo real y alertas

La columna derecha muestra las alertas activas (y una pestaña de resueltas) y la actividad en tiempo real: cada evento con hora, ícono, tipo, zona y ocupación, con filtros (Todos, Vehículos, Estados, Sistema). Las filas nuevas se resaltan un instante. Al hacer clic en una alerta se abre su detalle.

12.7 Páginas Sistema y Redis y BD

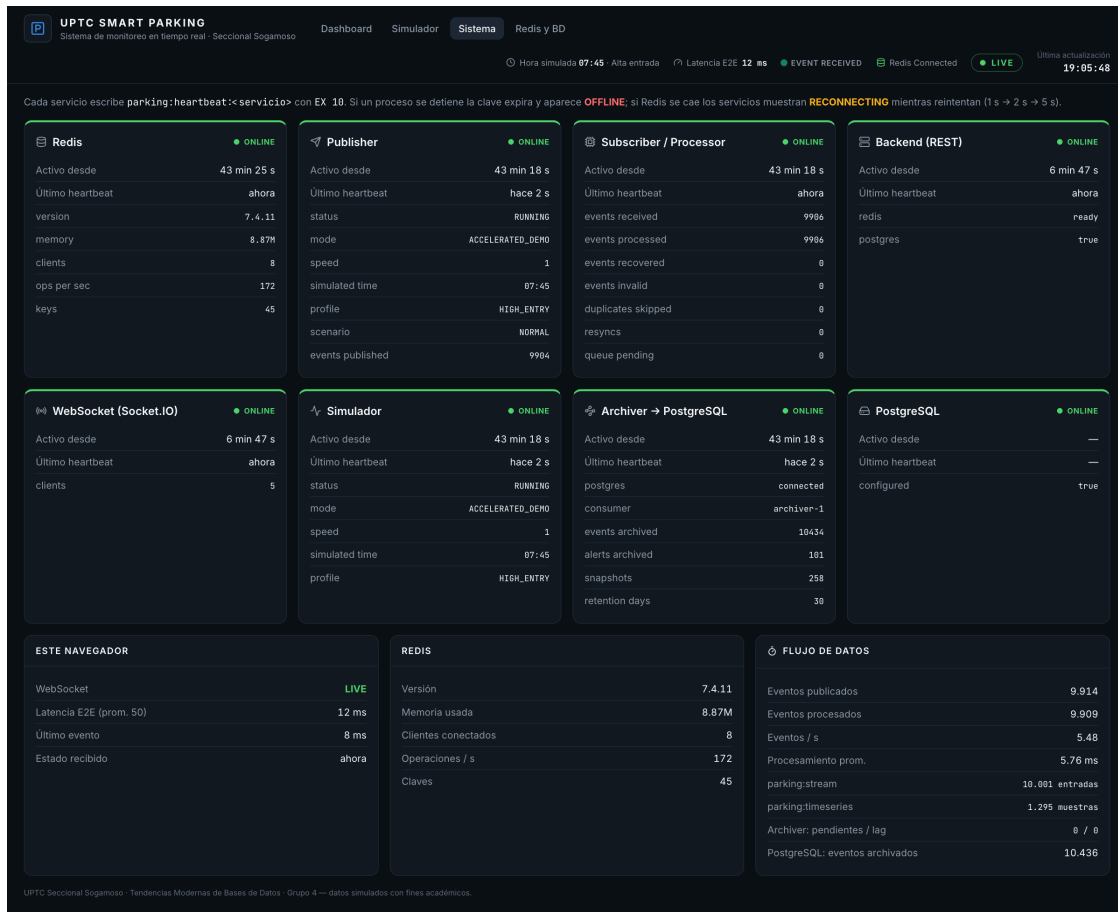


Figura 16. Página `/system`: cada servicio con su estado (ONLINE / OFFLINE / RECONNECTING / PAUSADO), tiempo activo, último heartbeat y datos internos.

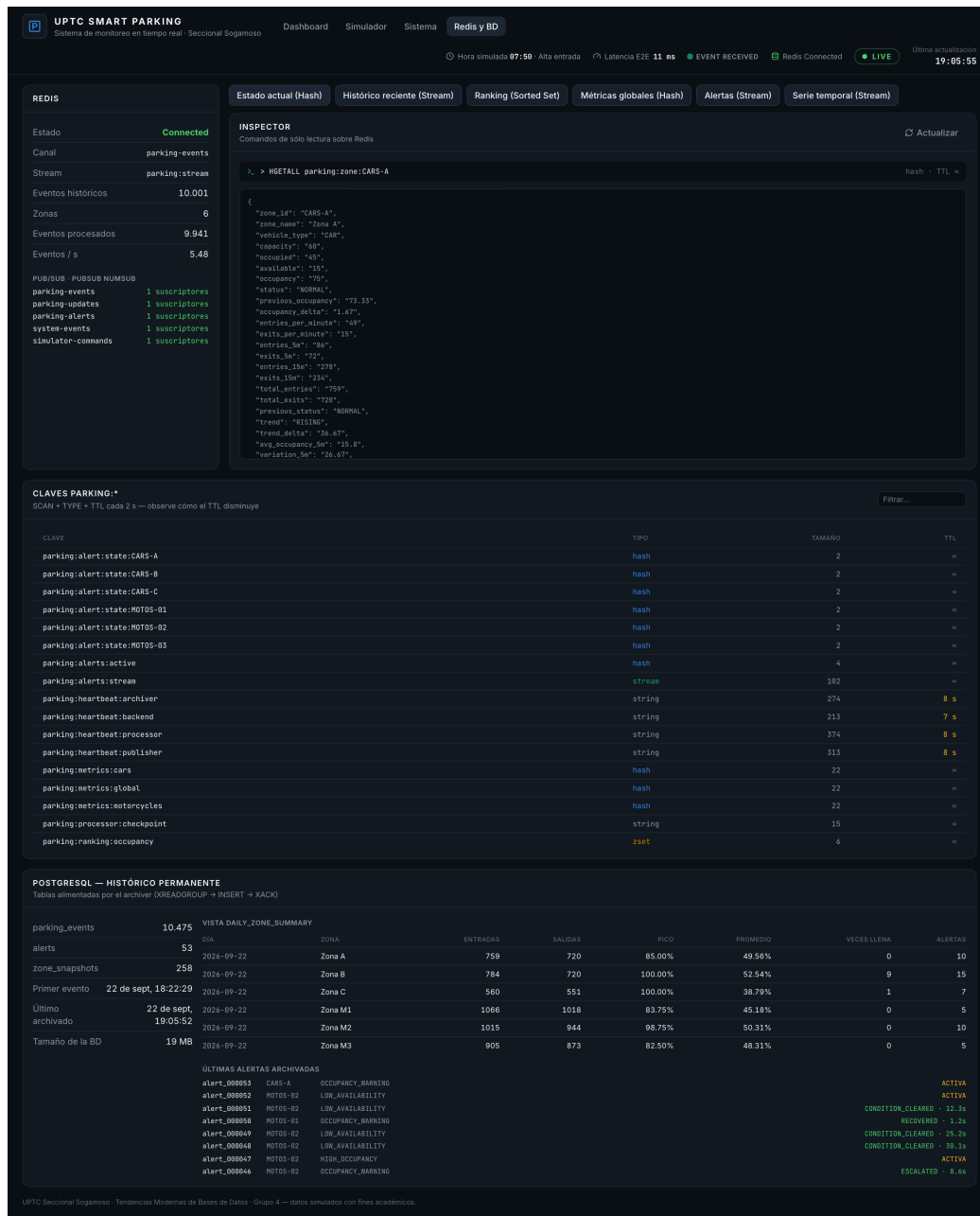


Figura 17. Página /debug: resumen de Redis, suscriptores por canal, inspector de comandos, claves con TTL en vivo y el histórico en PostgreSQL.

12.8 Cómo se actualiza sin recargar

1. Al abrir la página, el navegador pide el estado inicial por REST (zonas, métricas, ranking, historial, eventos, alertas).
2. Abre una conexión Socket.IO y desde ese momento recibe cada cambio empujado por el Backend.
3. Los mensajes se agrupan cada 120 ms y se aplican a un almacén de estado (Zustand); sólo se redibujan los componentes cuyo dato cambió.
4. Si la conexión se pierde, muestra ● RECONNECTING, reintenta cada 1 a 5 segundos y, al reconectar, vuelve a pedir el estado por REST para no quedar desactualizado.

13. PostgreSQL: el histórico permanente

13.1 Por qué además PostgreSQL

Redis responde "¿cómo está ahora?" y "¿cómo evolucionó en los últimos minutos?". Una solución real también necesita responder "¿qué pasó ayer, la semana pasada, en el semestre?" para reportes diarios, estadísticas mensuales, auditorías e incidentes. Esa información debe durar años y no cabe en memoria. Por eso el proyecto agrega **PostgreSQL** como almacenamiento permanente, **sin quitarle a Redis su papel central** en el tiempo real.

13.2 Usuario y base de datos

Parámetro	Valor
Usuario (rol)	<code>proyecto_electiva_1</code>
Contraseña	<code>Admin</code> (de desarrollo: cambiarla antes de publicar en una VPS)
Base de datos	<code>uptc_smart_parking</code>
Conexión desde el equipo	<code>postgres://proyecto_electiva_1:Admin@localhost:5432/uptc_smart_parking</code>
Conexión desde Docker	<code>postgres://proyecto_electiva_1:Admin@host.docker.internal:5432/uptc_smart_parking</code>

Se crean con `npm run db:setup`. El nombre del rol usa guiones bajos porque los espacios obligan a citar el identificador en SQL y rompen las URL de conexión.

13.3 El servicio archiver

El archiver lee `parking:stream` y `parking:alerts:stream` con un **consumer group** de Redis y copia cada entrada a PostgreSQL:

```
XGROUP CREATE parking:stream archiver 0 MKSTREAM
XREADGROUP GROUP archiver archiver-1 COUNT 200 BLOCK 5000 STREAMS parking:stream
parking:alerts:stream > >
INSERT INTO parking_events (...) VALUES (...) ON CONFLICT (stream_id) DO NOTHING
XACK parking:stream archiver <ids...>      ← sólo después de escribir en PostgreSQL
```

- Si el archiver o PostgreSQL se detienen, los eventos esperan en el Stream (el `*lag*` del grupo crece y se ve en `/system`) y se archivan al volver.
- Si se cae entre la lectura y la confirmación, las entradas quedan pendientes (PEL) y se reintentan al reiniciar.
- Nada se duplica: `parking_events` usa el ID del Stream como clave primaria y `alerts` usa (`id`, `raised_at`).
- Cada 60 s guarda además una **instantánea** del estado de cada zona.

13.4 Tablas

Tabla / vista	Contenido
<code>zones</code>	Catálogo de zonas (desde config/zones.json)
<code>parking_events</code>	Todos los eventos con su <code>stream_id</code> , tipo, zona, instante, ocupación y JSON completo. Restricción CHECK: <code>occupied ≤ capacity</code>
<code>alerts</code>	Alertas levantadas y resueltas, con motivo y hora de resolución
<code>zone_snapshots</code>	Instantánea del estado de cada zona cada 60 segundos
<code>daily_zone_summary</code> (vista)	Reporte diario por zona: entradas, salidas, pico, promedio, veces que se llenó y alertas

```
-- Reporte del día por zona
SELECT * FROM daily_zone_summary WHERE day = current_date ORDER BY zone_id;

-- ¿Cuántas veces se llenó cada zona esta semana?
SELECT entity_id, count(*) FROM parking_events
  WHERE event_type = 'PARKING_FULL' AND occurred_at > now() - interval '7 days' GROUP BY 1;

-- Duración promedio de las alertas por tipo
SELECT type, round(avg(extract(epoch FROM resolved_at - raised_at)), 1) AS segundos
  FROM alerts WHERE status = 'RESOLVED' GROUP BY type;
```

La retención es configurable (`ARCHIVE_RETENTION_DAYS`, 30 días por defecto): el archiver borra cada hora lo que supere ese plazo.

14. Tolerancia a fallos

Un sistema distribuido debe seguir funcionando cuando una de sus partes falla. La tabla resume qué ocurre ante cada falla y si se comprobó en vivo con el sistema en Docker.

Situación	Qué pasa	Probado
Se cae Redis	Todos los servicios reintentan a 1 s → 2 s → 5 s. La API responde 503; el dashboard muestra "Redis reconectando"; el Publisher deja de simular. Al volver todo se reconecta solo y el estado se conserva (RDB).	Sí: Redis detenido 8 s
Se cae el Processor	El Publisher registra receivers=0. Al volver, el Subscriber lee el Stream desde su checkpoint, recupera lo perdido y envía RESYNC.	Sí: 106 eventos recuperados
Se cae el archiver o PostgreSQL	Los eventos esperan en el Stream (lag). Al volver se archivan sin duplicar.	Sí
Se reinicia el Backend	El dashboard muestra RECONNECTING, reconecta solo y recarga el estado por REST.	Sí
Se reinicia el Publisher	Restaura reloj, escenario y ocupación desde Redis y continúa.	Sí
Llega un evento inválido	El Publisher no lo publica; el Subscriber descarta mensajes que no cumplen el esquema.	Pruebas unitarias
Un servicio muere sin avisar	Su heartbeat expira en ≤ 10 s y /system lo marca OFFLINE.	Sí

PARA LA EXPOSICIÓN

Demostración de resiliencia: ejecutar `docker compose stop processor`, mostrar que el tablero se congela y que el Publisher advierte `receivers=0`; esperar 15 segundos y ejecutar `docker compose start processor`. En los logs aparece "Recovered N events missed while offline from parking:stream" y el tablero se pone al día solo.

Esto demuestra en vivo la diferencia entre Pub/Sub (se pierde) y Streams (se conserva).

15. Estructura del código

El proyecto es un monorepo con espacios de trabajo npm. El paquete **shared** contiene el contrato común (tipos, esquemas, claves) que usan todos los servicios, de modo que un evento significa lo mismo en todas partes.

```

uptc-smart-parking/
├── apps/
│   ├── publisher/      simulation/ (simulador) · publisher/ (normaliza, valida, publica)
│   ├── processor/      subscriber/ · processor/ · metrics/ · alerts/ · redis/
│   ├── backend/        routes/ · controllers/ · services/ · websocket/ · redis/ · db/
│   ├── archiver/       consumer group Redis Streams → PostgreSQL
│   └── frontend/       components/ · pages/ · charts/ · hooks/ · stores/ · services/
├── packages/shared/    constants/ · schemas/ · types/ · domain/ · events/ · node/
├── config/             zones.json · thresholds.json · simulation.json
├── db/schema.sql       esquema de PostgreSQL
├── docker/            Dockerfiles · nginx.conf · redis.conf
├── docs/              documentación técnica
├── scripts/           setup-postgres · redis-cli · simulate-day · socket-probe · test-integration
├── tests/integration  pruebas con Redis real
└── docker-compose.yml · .env.example · README.md

```

15.1 Archivos clave y su responsabilidad

Archivo	Qué hace
<code>packages/shared/src/domain/occupancy.ts</code>	Reglas de negocio: ocupación, disponibles, estado, tendencia, límites (entrada en zona llena, salida en zona vacía)
<code>packages/shared/src/schemas/event.schema.ts</code>	Esquema Zod de los eventos y reglas de consistencia
<code>packages/shared/src/constants/redis-keys.ts</code>	Todas las claves Redis del sistema
<code>packages/shared/src/node/redis.ts</code>	Conexión a Redis con reconexión 1 s → 2 s → 5 s y heartbeats con TTL
<code>apps/publisher/src/simulation/parking-simulator.ts</code>	Ciclo de simulación: reloj, perfiles, planificación y aplicación de movimientos
<code>apps/publisher/src/simulation/scenario-manager.ts</code>	Escenarios NORMAL, HIGH_DEMAND, MASS_ENTRY, MASS_EXIT, NEAR_FULL, FULL, RECOVERY
<code>apps/publisher/src/publisher/event-normalizer.ts</code>	Lectura cruda del sensor → modelo común de eventos
<code>apps/publisher/src/publisher/publisher.service.ts</code>	XADD al Stream + PUBLISH al canal
<code>apps/processor/src/subscriber/event-subscriber.ts</code>	SUBSCRIBE, validación, cola serial y recuperación desde el Stream
<code>apps/processor/src/processor/event-processor.ts</code>	Los pasos del procesamiento de cada evento
<code>apps/processor/src/alerts/alert-engine.ts</code>	Máquina de estados de alertas (función pura, probada con pruebas unitarias)
<code>apps/processor/src/redis/state-repository.ts</code>	Acceso a Hashes, Sorted Sets y Streams del Processor
<code>apps/processor/src/metrics/timeseries-sampler.ts</code>	Muestra cada 2 s para las gráficas
<code>apps/backend/src/websocket/socket-gateway.ts</code>	Puente Redis → Socket.IO
<code>apps/archiver/src/index.ts</code>	Consumer group y escritura en PostgreSQL
<code>apps/frontend/src/hooks/useRealtime.ts</code>	Carga inicial por REST y actualizaciones por WebSocket

16. Instalación y ejecución

16.1 Requisitos

- **Docker Desktop** (o Docker Engine con Compose v2).
- **PostgreSQL** instalado en el equipo (opcional: sin él todo funciona excepto el histórico permanente; también puede usarse un PostgreSQL en contenedor).
- Para desarrollo local: **Node.js 22 o superior**.

16.2 Ejecución con Docker (recomendada)

```
git clone <repositorio>
cd uptc-smart-parking
cp .env.example .env
npm run db:setup # crea el usuario y la base en PostgreSQL (una vez)
docker compose up --build # levanta los 6 servicios

# Opcionales
docker compose --profile insight up --build # + RedisInsight en http://localhost:5540
docker compose --profile local-db up --build # + PostgreSQL en contenedor (puerto 5433)
```

Puerto	Servicio
5173	Dashboard → http://localhost:5173
3000	Backend → http://localhost:3000/api/health
6379	Redis
5540	RedisInsight (perfil insight)
5432 / 5433	PostgreSQL local / en contenedor

16.3 Modo desarrollo

```
npm install
cp .env.local.example .env.local # REDIS_HOST=localhost para ejecutar fuera de Docker
docker compose up -d redis
npm run dev # publisher, processor, backend, archiver y Vite con
recarga automática
```

16.4 Configuración

Archivo / variable	Para qué
<code>config/zones.json</code>	Zonas: id, nombre, tipo, capacidad y factor de demanda
<code>config/thresholds.json</code>	Umbral de estados y alertas, tendencia, histéresis, cooldown
<code>config/simulation.json</code>	Perfiles de demanda y parámetros de cada escenario
<code>SIMULATION_INTERVAL_MS</code>	Intervalo entre ciclos (5000 por defecto; 2000 para la demostración)
<code>SIMULATION_MODE</code>	ACCELERATED_DEMO o REAL_TIME_PROFILE
<code>STREAM_MAX_LENGTH</code>	Tamaño máximo de parking:stream (10000)
<code>WARNING_OCCUPANCY · CRITICAL_OCCUPANCY · LOW_AVAILABILITY</code>	Sobrescriben los umbrales (80 · 90 · 5)
<code>ALERT_COOLDOWN_SECONDS</code>	TTL del cooldown de alertas (30)
<code>DATABASE_URL</code>	Conexión a PostgreSQL
<code>DEMO_CONTROL_TOKEN</code>	Protege el panel de simulación (vacío = sin token)

Cambiar zonas o umbrales sólo requiere editar el archivo y reiniciar los servicios (`docker compose restart`): la carpeta `config/` está montada como volumen.

17. Pruebas y resultados

17.1 Pruebas unitarias (68)

Se ejecutan con `npm test` y cubren, entre otras:

- Cálculo de ocupación y disponibilidad: 80 de capacidad y 73 ocupados → 7 disponibles y 91,25 %.
- Clasificación en cada umbral: 59,99 % LOW · 60 % NORMAL · 80 % WARNING · 90 % CRITICAL · 100 % FULL.
- Límites: occupied nunca supera la capacidad ni es negativo; entrada en zona llena (80/80 sigue en 80); salida en zona vacía (0/80 sigue en 0).
- Tendencia RISING / FALLING / STABLE.
- Perfiles por hora, continuidad del simulador durante un día completo, escenarios FULL, NEAR_FULL, RECOVERY y MASS_ENTRY, reproducibilidad con semilla.
- Normalización y validación de eventos (rechazo de eventos inconsistentes).
- Métricas globales, por tipo, ranking y ventanas.
- Alertas: secuencia completa, sin duplicados, histéresis, cooldown, disponibilidad baja, crecimiento inusual, alertas por tipo de parqueadero.

17.2 Pruebas de integración con Redis real (4)

`npm run test:integration` levanta un Redis desechable en el puerto 6380 y verifica: **Pub/Sub** (Publisher → Redis → Subscriber), **Stream** con límite MAXLEN, **Hash + Sorted Set + alertas + TTL** con la secuencia 77,5 → 82,5 → 91,25 → 100 → 75 %, y **recuperación** tras la desconexión del Subscriber.

17.3 Pruebas del sistema en ejecución

Prueba	Resultado
Tiempo real sin recargar	Eventos, métricas y alertas llegan por WebSocket; latencia E2E de 9 a 13 ms
Secuencia de alertas en la Zona C (Cerca de lleno → Llenar zona → Recuperación)	OCCUPANCY_WARNING → OCCUPANCY_CRITICAL → PARKING_FULL → desescalado → ZONE_RECOVERED
Processor detenido 10 s	receivers=0 en el Publisher; al volver recuperó 106 eventos del Stream
Redis detenido 8 s	Reintentos 1 s → 2 s → 5 s; API 503 durante la caída; todo volvió a healthy y conservó el estado
Archivo en PostgreSQL	Lag 0; más de 10 000 eventos y 100 alertas archivados; reporte diario correcto

17.4 Mediciones

Medida	Valor
Latencia de extremo a extremo (sensor → navegador)	≈ 9–13 ms (p95 ≈ 16–19 ms)
Procesamiento de un evento en el Processor	≈ 4–6 ms
Eventos por día simulado	≈ 3 400
Eventos por segundo en hora pico	≈ 5–7
Memoria de Redis con el Stream lleno (10 000 eventos)	≈ 9 MB
Ocupación máxima en escenario NORMAL	78–86 % (sin alertas críticas)

18. Guía para la exposición

18.1 Preparación (antes de entrar)

1. En `.env` poner `SIMULATION_INTERVAL_MS=2000` para que haya más movimiento visible.
2. Ejecutar `docker compose --profile insight up --build -d`.
3. Abrir el dashboard (<http://localhost:5173>), el panel `/simulator`, la página `/debug` (o RedisInsight en <http://localhost:5540>) y dos terminales con `docker compose logs -f publisher` y `docker compose logs -f processor`.
4. Unos 30 segundos antes de empezar, en `/simulator`, presionar **Reiniciar día** para arrancar desde las 06:30 y mostrar la llegada masiva.

18.2 Guion paso a paso

#	Qué hacer	Qué decir
1	—	"Monitoreamos en tiempo real la ocupación de los parqueaderos de carros y motos" (30 s).
2	Mostrar la figura de arquitectura o el panel "Recorrido de un evento"	Simulador → Publisher → Redis → Processor → Backend → Dashboard; cada servicio es independiente.
3	<code>/system</code> o RedisInsight	Redis funcionando: versión, memoria, claves, clientes.
4	Terminal del Publisher	"Aquí se publican los eventos: <code>VEHICLE_ENTERED ... receivers=1</code> ".
5	<code>./scripts/redis-cli.sh SUBSCRIBE parking-events</code>	Pub/Sub: los mensajes llegan en vivo.
6	<code>/debug</code> → Histórico reciente (Stream)	<code>XREVRANGE parking:stream</code> : los eventos quedan guardados.
7	<code>/debug</code> → Estado actual (Hash)	<code>HGETALL parking:zone:CARS-A</code> : la ficha actual de la zona.
8	Dashboard	● LIVE y "Redis Connected".
9	<code>/simulator</code> → Zona A → Forzar <code>VEHICLE_ENTERED</code>	La tarjeta suma 1, destella, cambia el KPI y aparece en la actividad. Sin F5.
10	Gráfica → Zona específica: Zona A	El nuevo valor entra en la serie.
11	<code>/simulator</code> → Zona C → Cerca de lleno	Aparecen <code>OCCUPANCY_WARNING</code> y <code>OCCUPANCY_CRITICAL</code> y sus alertas.
12	Llenar zona	<code>PARKING_FULL</code> : tarjeta roja, 0 disponibles, alerta <code>CRITICAL</code> .
13	Recuperación	Salidas progresivas y <code>ZONE_RECOVERED</code> ; la alerta pasa a Resueltas.
14	<code>HGETALL parking:zone:CARS-C</code>	El estado actual refleja la recuperación.
15	<code>XREVRANGE parking:stream + - COUNT 5</code>	El histórico muestra cómo llegó ahí.
16	—	"Hash = estado actual; Stream = histórico reciente".
Extra	<code>docker compose stop processor</code> y luego <code>start</code>	Pub/Sub pierde, el Stream conserva: "Recovered N events".

18.3 Reparto sugerido entre los integrantes

Bloque	Temas	Capítulos
1. Problema y arquitectura	Problema, objetivos, arquitectura orientada a eventos, componentes, recorrido de un evento	1 – 5
2. Simulador y Publisher	Continuidad, perfiles, escenarios, normalización, validación, Pub/Sub	5, 6, 7.3
3. Redis y procesamiento	Streams, Hashes, Sorted Sets, TTL, estado vs histórico, Processor, métricas, alertas	7 – 10
4. Dashboard, demo y cierre	Dashboard, demostración en vivo, tolerancia a fallos, PostgreSQL, limitaciones y conclusiones	11 – 21

Aunque cada uno exponga un bloque, **todos deben poder responder** sobre la arquitectura, Pub/Sub, Streams, Hashes, estado vs histórico, simulación, métricas, alertas y limitaciones (capítulo 19).

18.4 Si algo falla en vivo

Síntoma	Solución
El dashboard muestra RECONNECTING	<code>docker compose ps</code> ; si el backend no está arriba: <code>docker compose up -d backend</code>
No llegan eventos	En /simulator verificar que el estado sea Running (no Paused); si no, Reanudar
"Publisher no está escuchando órdenes"	<code>docker compose restart publisher</code>
PostgreSQL OFFLINE en /system	Verificar que PostgreSQL esté iniciado en el equipo; el resto del sistema sigue funcionando
Puerto 5173 o 3000 ocupado	Cambiar FRONTEND_PORT o BACKEND_PORT en .env
Quiero empezar de cero	<code>./scripts/redis-cli.sh FLUSHALL</code> y <code>docker compose restart</code>

19. Preguntas de sustentación

Respuestas preparadas para las preguntas que el enunciado anticipa. Todos los integrantes deben conocerlas.

¿Por qué utilizar Redis?

Porque necesitamos procesar información dinámica con baja latencia. Redis guarda los datos en memoria y además ofrece Pub/Sub, Streams, Hashes y Sorted Sets, que encajan con una arquitectura orientada a eventos. En el sistema, procesar un evento toma unos 4 ms y el recorrido completo hasta el navegador unos 13 ms.

¿Qué información se mantiene en memoria?

El estado actual de las zonas (Hashes), las métricas agregadas, las alertas activas, el histórico reciente (Streams), el ranking de ocupación (Sorted Set), las ventanas de entradas y salidas y la información temporal con TTL (heartbeats y cooldowns).

¿Qué representa el estado actual?

La medición más reciente conocida de una zona: capacidad, ocupados, disponibles, porcentaje, estado, tendencia y última actualización. Se consulta con `HGETALL parking:zone:<id>`.

¿Qué representa el histórico?

La secuencia reciente de entradas, salidas, cambios de estado y alertas que permite analizar cómo evolucionó el estacionamiento y construir las gráficas. Está en `parking:stream` y `parking:timeseries`.

¿Por qué utilizar Hashes?

Porque una zona tiene muchos atributos relacionados; el Hash permite leerlos juntos con un solo comando y modificar sólo los que cambian.

¿Por qué utilizar Streams?

Para conservar un histórico reciente que se puede consultar después de publicado: recuperar eventos perdidos, dibujar las gráficas y archivar en PostgreSQL con consumer groups.

¿Cuál es la función del Publisher?

Obtener las lecturas del simulador, normalizarlas al formato común, validarlas con Zod y publicarlas en Redis (XADD al Stream y PUBLISH al canal).

¿Cuál es la función del Subscriber?

Escuchar los eventos del canal `parking-events` y entregarlos en orden al Processor; al reconectarse, recuperar del Stream lo que se perdió.

¿Qué información viaja por Pub/Sub?

Entradas y salidas en JSON, estados procesados y métricas, alertas, eventos de la simulación y las órdenes del panel de simulación.

¿Qué sucede si el Subscriber se desconecta?

Los mensajes Pub/Sub publicados durante la desconexión no le llegan. Por eso cada evento también se guarda en parking:stream: al volver, el Subscriber lee desde su último checkpoint y recupera lo perdido. Lo demostramos en vivo.

¿Qué sucede si Redis se reinicia?

Depende de la persistencia. Usamos instantáneas RDB en un volumen, así que el estado se recupera. Además los sensores envían conteos absolutos: aunque Redis empezara vacío, el primer evento de cada zona la recalibra. En producción podría activarse AOF.

¿Cómo controlan el crecimiento de los datos?

Streams con MAXLEN ~, ventanas recortadas a 15 minutos con EXPIRE, TTL para la información temporal, eliminación de las alertas resueltas del Hash de activas, límite de memoria con volatile-lru y retención de 30 días en PostgreSQL.

¿Cómo generan los eventos?

Con un simulador que conserva el estado anterior de cada zona y genera entradas o salidas según perfiles de demanda por hora (llegadas tipo Poisson y probabilidades de entrada y salida), respetando siempre la capacidad. No son números aleatorios independientes.

¿Qué métricas calculan?

Ocupación por zona, global, de carros y de motos; espacios disponibles; entradas y salidas por minuto y en 5 y 15 minutos; variación; tendencia; zona más ocupada; zonas críticas; promedios por ventana.

¿Cómo generan alertas?

El Processor compara las métricas con umbrales configurables usando una máquina de estados por zona: 80 % advertencia, 90 % crítica, 100 % PARKING_FULL, 5 espacios disponibilidad baja, +20 pp en 6 s crecimiento inusual. Usa histéresis y cooldown para no repetirlas y genera ZONE_RECOVERED al recuperarse.

¿Qué parte funciona en tiempo real?

Todo el recorrido: el Publisher publica en Redis, el Subscriber recibe el evento, el Processor actualiza el estado y el Backend lo transmite por WebSocket al dashboard, que cambia sin recargar.

¿Qué ventajas tiene una base en memoria?

Baja latencia, alta velocidad de lectura y escritura, estructuras optimizadas y mecanismos de eventos integrados.

¿Qué limitaciones tiene Redis?

Depende de la memoria RAM, el histórico permanente es limitado, la persistencia es distinta a la de una base relacional y Pub/Sub no conserva mensajes.

¿Qué pasa si aumentan los sensores?

Se distribuye la generación entre varios Publishers y el procesamiento entre varios consumidores (consumer groups particionados por zona).

¿Cómo soportar múltiples Publishers?

Cada Publisher representaría una zona, un parqueadero o un grupo de sensores; todos publican en los mismos canales y Stream, y los IDs salen de un contador atómico de Redis, así que no chocan.

¿Qué componentes pueden escalar horizontalmente?

Publishers, Processors, Backends y Subscribers. Redis puede escalar con réplicas, Sentinel o Cluster.

¿Por qué además PostgreSQL?

Redis guarda el presente y el pasado reciente; PostgreSQL guarda lo que debe durar (reportes diarios, auditoría de alertas). El archiver lo alimenta con un consumer group sin afectar el tiempo real.

20. Limitaciones, dificultades y mejoras

20.1 Limitaciones

- La memoria RAM es más limitada que el disco: Redis no es adecuado para guardar años de histórico (por eso PostgreSQL).
- Con RDB, una caída abrupta de Redis puede perder los últimos minutos (AOF reduciría la pérdida a ≈ 1 s).
- Pub/Sub no conserva mensajes para consumidores desconectados (mitigado con Streams y checkpoint).
- Hay un solo Processor que procesa en serie; para cargas mucho mayores habría que particionar.
- Las ventanas se miden en tiempo real: a velocidad x5 la compresión del tiempo puede hacer que el pico matutino se detecte como crecimiento inusual.
- Los datos son simulados; zonas, capacidades y perfiles no corresponden a la universidad.

20.2 Dificultades encontradas y cómo se resolvieron

Dificultad	Solución
Alertas "parpadeantes": una zona de 50 espacios que oscila entre 39 y 40 vehículos cruzaba el 80 % con cada auto.	Histéresis de 2 pp, cooldown con TTL y eventos de advertencia derivados de la máquina de alertas.
El pico de la mañana acelerado 60 veces parecía "crecimiento inusual".	Se calibró la ventana a 6 s: el pico sube ≈ 7 pp y una entrada masiva ≈ 30 pp.
El primer evento de cada zona saltaba de 0 a la ocupación real y disparaba alertas falsas.	Se trata como calibración.
Los canales Pub/Sub son globales a toda la instancia de Redis (no dependen del número de base): las pruebas contaminaban el sistema en ejecución.	Las pruebas de integración usan un Redis desechable aparte.
Al vaciar Redis el contador de alertas se reinicia y PostgreSQL descartaba alertas con IDs repetidos.	La clave de la tabla pasó a ser (id, raised_at), con migración automática.

20.3 Mejoras futuras

- Sensores reales (MQTT o LoRaWAN) publicando en Redis Streams.
- Varios Processors con consumer groups particionados por zona; Redis Sentinel o Cluster.
- Predicción de ocupación con series de tiempo y recomendación de la zona más libre.
- Aplicación móvil y pantallas en las entradas del campus con la disponibilidad en vivo.
- Autenticación y roles (vigilancia, administración).
- Particionado mensual en PostgreSQL y un tablero de analítica histórica.

21. Conclusiones

- **Redis puede cumplir varios papeles a la vez:** bus de eventos (Pub/Sub), registro de eventos (Streams), base de datos del estado actual (Hashes), índice ordenado (Sorted Sets) y almacén de datos temporales (TTL). Cada estructura resolvió un problema concreto del sistema.
- **Estado actual e histórico reciente son cosas distintas:** el Hash responde en una sola lectura cómo está una zona; el Stream permite reconstruir cómo llegó ahí y recuperar eventos perdidos.
- **Pub/Sub y Streams se complementan:** el primero da inmediatez, el segundo durabilidad; juntos permiten un sistema en tiempo real que tolera la caída de sus consumidores (demostrado con 106 eventos recuperados).
- **Una simulación con continuidad y perfiles** produce datos creíbles y permite demostrar condiciones normales, críticas y de recuperación de forma controlada.
- **Las alertas útiles necesitan estado** (niveles, histéresis, cooldown), no sólo comparar contra un umbral.
- **Una base en memoria no reemplaza a una base histórica:** PostgreSQL complementa a Redis para la información que debe durar.
- El sistema cumple el objetivo: un evento viaja del sensor a la pantalla en unos 13 ms y el tablero cambia sin recargar.

Anexo A. Glosario

Términos técnicos utilizados en el documento, en orden alfabético.

Término	Definición
AOF / RDB	Mecanismos de persistencia de Redis: registro de cada escritura (AOF) o instantáneas periódicas (RDB)
Checkpoint	ID del último evento procesado; permite retomar desde ahí
Consumer group	Grupo de lectores de un Stream que se reparten las entradas y confirman cada una (XACK)
Cooldown	Tiempo de espera (con TTL) en el que no se repite una alerta
Evento derivado	Evento calculado por el Processor a partir del estado (PARKING_FULL, ZONE_RECOVERED...)
Hash	Estructura de Redis con varios campos bajo una clave (ficha)
Heartbeat	Señal periódica "estoy vivo" que cada servicio escribe con TTL
Histéresis	Margen para no cambiar de nivel con cada pequeña oscilación
Latencia E2E	Tiempo desde que se genera el evento hasta que llega al navegador
MAXLEN ~	Límite aproximado de entradas de un Stream
Pipeline	Envío de varios comandos Redis en un solo viaje de red
Pub/Sub	Mensajería publicar/suscribir de Redis: entrega inmediata sin almacenamiento
Poisson	Distribución que modela llegadas aleatorias con una tasa promedio
Sorted Set	Conjunto de Redis ordenado por puntaje
Stream	Registro de Redis de solo agregar, con IDs basados en el tiempo
TTL	Tiempo de vida de una clave; al cumplirse Redis la borra
WebSocket / Socket.IO	Conexión permanente navegador-servidor que permite enviar datos sin que el navegador pregunte
Zod	Librería de validación de datos con esquemas

Anexo B. Comandos Redis útiles para la demostración

Con `./scripts/redis-cli.sh` se ejecuta `redis-cli` dentro del contenedor:

```
# Estado actual
HGETALL parking:zone:MOTOS-02
HGETALL parking:metrics:global

# Histórico reciente
XLEN parking:stream
XREVRANGE parking:stream + - COUNT 5
XRANGE parking:stream - +

# Pub/Sub en vivo (Ctrl+C para salir)
SUBSCRIBE parking-events
PUBSUB NUMSUB parking-events parking-updates parking-alerts

# Ranking
ZREVRANGE parking:ranking:occupancy 0 -1 WITHSCORES
ZREVRANGE parking:ranking:occupancy 0 0 WITHSCORES

# Ventanas y TTL
ZCOUNT parking>window:entries:CARS-A (1790121484759 +inf
TTL parking:heartbeat:processor
KEYS parking:alert:cooldown:*

# Consumer group del archiver
XINFO GROUPS parking:stream
```

Anexo C. Cumplimiento de los requisitos

Correspondencia entre los requisitos del enunciado (secciones 174, 175 y 191) y la implementación, con el lugar donde puede verificarse cada uno.

Requisito del enunciado	Cómo se cumple	Dónde verlo
Redis, Publisher, Subscriber, Processor funcionando	Servicios independientes en Docker	/system
Pub/Sub	5 canales	Cap. 7.3 · /debug
Al menos dos estructuras Redis	Pub/Sub, Streams, Hashes, Sorted Sets, TTL	Cap. 7
Datos continuos	Simulador con ciclos cada 2–5 s	Actividad en tiempo real
Procesamiento y ≥ 2 métricas derivadas	11 métricas + ventanas	Cap. 9
≥ 1 alerta	7 tipos con recuperación	Cap. 10
Dashboard sin refresh	Socket.IO	Cap. 12
≥ 2 gráficas	3 gráficas	Cap. 12.5
Estado actual e histórico reciente	Hash vs Stream	Cap. 7.7 · detalle de zona
Simulador	Perfiles, modos y 7 escenarios	Cap. 6 · /simulator
Arquitectura documentada	Este documento, README y docs/	—
Streams con crecimiento controlado	MAXLEN ~	Cap. 7.11
TTL	Heartbeats, cooldowns, métricas temporales, ventanas	/debug (TTL en vivo)
Ventanas temporales, ranking, crecimiento inusual	1/5/15 min, Sorted Set, alerta UNUSUAL	Cap. 7.8 y 10
Docker y ejecución reproducible	docker compose up --build	Cap. 16
Reconexión Redis y WebSocket	1 s $\rightarrow$ 2 s $\rightarrow$ 5 s; Socket.IO	Cap. 14
Logs estructurados	Formato hora · nivel · servicio · mensaje	docker compose logs
Métricas internas y latencia E2E	Eventos/s, ms por evento, latencia	Encabezado · /system
Recuperación de alertas	ZONE_RECOVERED + RESOLVED	Cap. 10.4
Pruebas	72 automatizadas	Cap. 17

Anexo D. Documentación complementaria en el repositorio

El repositorio incluye, además de este documento, la siguiente documentación técnica en formato Markdown:

Archivo	Contenido
README.md	Instalación, ejecución, configuración, API y demostración resumida
docs/architecture.md	Arquitectura, diagramas y escalabilidad
docs/redis-design.md	Diseño completo de claves y estructuras Redis
docs/event-model.md	Esquema de eventos, alertas y mensajes
docs/simulator.md	Modelo de simulación, perfiles y escenarios
docs/database.md	Esquema de PostgreSQL, archiver y retención
docs/technical-report.md	Documento técnico con la estructura de 39 secciones del enunciado
docs/presentation.md	Esquema de diapositivas, guion y checklists

— Fin del documento —